



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF SOFTWARE TECHNOLOGY AND METHODOLOGY

Formal Verification of Questionnaire Logic

Designing and Checking Surveys with SMT-Based Analysis

Supervisor:

Péter Bereczky

Assistant Lecturer

Author:

Péter Sághegyi

Computer Science MSc

Budapest, 2025

Abstract

This thesis presents a formal, declarative model for questionnaire specification that fundamentally differs from traditional flow-based approaches. Rather than viewing questionnaires as sequential execution paths, we formalize them as collections of items with preconditions (Boolean predicates determining when questions appear) and postconditions (constraints on acceptable responses). This declarative paradigm, inspired by Hoare Logic, treats the questionnaire as a set of logical relationships where the actual flow emerges dynamically based on respondent answers. Our static analysis investigates all possible paths simultaneously through constraint satisfaction, reasoning about relationships between immutable states rather than simulating execution through mutable values.

Our implementation leverages the Z3 SMT solver to provide mathematical guarantees about questionnaire correctness. Despite questionnaire consistency checking being NP-complete (Willenborg, 1995), modern SMT solvers handle real-world questionnaires effectively. This enables validation of questionnaires containing hundreds of interconnected questions with complex conditional dependencies—a level of complexity that exceeds human comprehension and often makes traditional validation approaches computationally intractable.

Contents

1	Introduction	3
1.1	Key Concepts in Modern Questionnaires	3
1.2	The Problem of AI-Generated Questionnaire Logic	4
1.3	Historical Development of Questionnaire Logic Analysis	6
1.4	The Declarative Revolution: From Imperative to Functional Paradigms	9
1.5	Contributions	10
1.6	Thesis Organization	11
2	Related Work	12
2.1	Satisfiability Modulo Theories: The Enabling Technology	12
2.2	Bridging Questionnaire Validation and SMT Solving	14
2.3	Formal Methods in Software Verification	16
3	Questionnaire Definition	17
3.1	Axiomatic Building Blocks	17
3.2	Questionnaire Structure	19
3.3	Precondition Properties	21
3.4	Postcondition Properties	22
3.5	Implementation via Python	25
3.6	Dependency Graph and Cycle Test	26
3.7	Extension to Complex Item Types	30
4	Questionnaire Compilation	34
4.1	Single Static Assignment (SSA)	37
4.2	Conditional Branching Transformation	38
4.3	Loop Unrolling	40
4.4	Integration with Questionnaire Logic	42
4.5	Z3 Compilation for Complex Types	45

4.6	Compiler Characteristics	50
5	Comprehensive Questionnaire Validation	52
5.1	Setup	52
5.2	Key Relations	53
5.3	Path-Based Validation	56
5.4	Validation Hierarchy	59
6	Conclusion	62
6.1	Summary of Contributions	62
6.2	Impact and Significance	63
6.3	Limitations and Challenges	65
6.4	Future Research Directions	65
6.5	Final Thoughts	67
	Bibliography	68
A	Practical Examples of Complex Type Questionnaires	72
A.1	Employee Satisfaction Survey	72
A.2	Product Comparison Matrix	74
A.3	Market Research Survey with Complex Dependencies	76

Chapter 1

Introduction

The proliferation of artificial intelligence in survey design and questionnaire generation has created new opportunities for automated market research, social studies, and data collection [1, 2]. AI-powered systems can now generate sophisticated questionnaires with complex conditional logic, branching paths, and interconnected question dependencies. However, this automation introduces a critical challenge: ensuring the logical consistency and completeness of AI-generated questionnaires [1, 3].

1.1 Key Concepts in Modern Questionnaires

Before discussing the challenges of AI-generated questionnaires, let us briefly introduce some key concepts that distinguish modern dynamic surveys from traditional paper forms:

Preconditions (also called **dependencies**) occur when one question’s appearance or behavior depends on answers given earlier. A question about “What brand is your car?” naturally depends on a positive answer to car ownership—it would make no sense to ask non-car owners about their car’s brand.

Branching logic (also called **skip logic**) refers to the ability of a questionnaire to take different paths based on previous answers. The term “skip” originates from the sequential nature of traditional questionnaires: when questions are listed in order, irrelevant ones must be “skipped” based on previous responses. For example, if someone answers “No” to “Do you own a car?”, they skip all car-related questions and jump directly to questions about public transportation. Technically, this is im-

plemented by adding the negation of the branching condition to the preconditions of the skipped questions—ensuring they are never shown to respondents who don’t meet the criteria.

Question accessibility means that every question in the survey should be reachable through at least one valid path of answers. An inaccessible question is like a room in a building with no doors—it exists in the questionnaire but no respondent can ever reach it, making it useless.

Postconditions (also called **edit rules**) are constraints that validate answers and ensure logical consistency. They serve two key purposes: First, they explicitly check whether a given answer makes sense in the context of previous responses—for instance, when collecting household financial data, verifying that the reported total family income is not less than the sum of all individual family members’ incomes plus any additional support payments. Second, postconditions can dynamically narrow the acceptable range of answers based on prior information. For example, while age might generally accept values from 1 to 100, if someone previously indicated they consume alcohol, a postcondition could enforce that their age must be at least 18 (or 21, depending on the country they selected in an earlier question to include more dynamic validation rules).

Logical contradictions arise when the rules governing questions conflict with each other, creating impossible situations. Imagine a survey that only shows a question to people who are both “under 18” and “have 20+ years of work experience”—no one can satisfy both conditions simultaneously, so the question becomes unreachable due to contradictory requirements.

These concepts, while powerful for creating intelligent and adaptive surveys, introduce complexity that traditional validation methods struggle to handle, especially when AI systems generate questionnaires automatically.

1.2 The Problem of AI-Generated Questionnaire Logic

Modern questionnaire systems, particularly those enhanced with artificial intelligence, can automatically generate surveys with conditional logic of unprecedented complexity [4]. While traditional manually-designed surveys remain manageable for

human review, AI-generated questionnaires present a fundamentally different challenge:

- **Complexity Beyond Human Comprehension:** AI can generate questionnaires with hundreds of questions containing intricate interdependencies that no human could fully analyze or verify manually.
- **Exponential Path Proliferation:** With each conditional branch, the number of possible paths through the questionnaire grows exponentially, quickly exceeding what even AI systems can exhaustively validate through simulation.
- **Hidden Logical Errors:** Complex preconditions involving dozens of variables can create subtle contradictions that only manifest in specific, hard-to-predict response combinations.
- **Emergent Circular Dependencies:** In questionnaires with 50+ questions, circular dependencies can emerge through chains of conditions that are practically impossible to detect through manual inspection.
- **Scale-Induced Validation Failure:** Traditional validation approaches that work for 10-20 question surveys become computationally intractable or unreliable when applied to AI-generated surveys with 100+ questions and complex conditional logic.

The core challenge is not merely that these problematic scenarios exist, but that automated generation creates questionnaires whose logical correctness cannot be verified by human review or even by the AI systems that created them. Beyond a certain threshold of questions and conditional complexity, detecting logical or flow errors becomes a genuine computational challenge that requires formal mathematical approaches [5].

1.3 Historical Development of Questionnaire Logic Analysis

The challenge of validating questionnaire logic is not new, though modern AI systems have amplified its importance. The formal study of questionnaire structure has evolved over six decades, progressing from theoretical foundations to practical tools and computational complexity results.

1.3.1 Early Foundations (1965-1976)

The formal analysis of questionnaires began with Picard’s theoretical work [6], which provided the earliest mathematical treatment of questionnaire structure. Picard’s *Théorie des questionnaires* established the conceptual foundation for viewing questionnaires as formal systems with logical relationships between questions.

Building on this theoretical groundwork, Fellegi and Holt [7] introduced a systematic approach to **edit rules**—consistency constraints that survey responses must satisfy. Their fundamental principle was that edit rules must be internally consistent: there must exist at least one response that satisfies all constraints simultaneously. This principle anticipated what we now formalize as the global satisfiability requirement for questionnaire logic. While Fellegi and Holt focused on post-collection data validation and error correction, their work established the critical concept that contradictory constraints render a questionnaire problematic.

1.3.2 Formalization and Complexity (1988-1995)

Willenborg’s dissertation [8] marked a significant advance by introducing a formal definition of questionnaire logical structure, distinguishing between:

- **Routing structure:** Conditional logic determining which questions are asked of which respondents
- **Edit structure:** Consistency rules that responses must satisfy

This formalization enabled Willenborg to propose automated checks for logical correctness. In subsequent work, Willenborg [5] proved that checking a questionnaire’s combined routing and edit logic for consistency is **NP-complete**, establish-

ing that no polynomial-time algorithm can solve the general problem. This complexity result underscores why sophisticated automated reasoning techniques are necessary for questionnaire validation.

1.3.3 Graph-Based Approaches (1988-2015)

Recognizing that questionnaire logic exhibits natural graph structure, researchers developed graph-based representations. Fagan and Greenberg [9] pioneered the use of graph theory for analyzing skip patterns in census questionnaires, representing questions as nodes and conditional transitions (skips) as directed edges.

Bethlehem and Hundepool [10] developed TADEQ (Tool for the Analysis of Electronic Questionnaires), which formalized properties that questionnaire graphs should satisfy:

- Single start node and end node
- Directed edges representing conditional transitions
- Preferably acyclic structure (no circular dependencies)

TADEQ distinguished between *logically possible routes* through a questionnaire and *incorrect routes* that violate design intent, providing documentation and visualization tools for questionnaire designers.

Elliott [11] advanced graph-based validation by deriving a *basis set of paths* through surveys to use as test cases, ensuring all conditional branches are tested. Elliott’s work explicitly handles questionnaire features such as multiple edges between nodes (different answers leading to the same next question) and even certain acceptable loops for error-checking revisions.

Schiopu-Kratina et al. [12] refined the graph formalism by emphasizing that questionnaire graphs require augmentation beyond standard graph theory: edges must be labeled with the conditions under which transitions occur. In pure graph theory, an edge either exists or not; in questionnaires, edges are *conditional* on specific responses. Their work stressed ensuring every question lies on some “logically possible route” and identifying “empty paths” (unreachable sequences).

1.3.4 Path Enumeration and Modern Approaches (2008-2019)

Manski and Molinari [13] introduced a decision-theoretic perspective, formalizing skip logic as a design choice balancing information gain against survey cost and respondent burden. Their work provides theoretical justification for *why* skip logic is used: it represents a rational trade-off in questionnaire design.

Feeney and Feeney [14] presented one of the most recent comprehensive treatments, arguing that logical structure is critical yet often neglected in questionnaire design. They developed computer-readable representations and algorithms to enumerate all questionnaire paths, determining which respondents are eligible for which questions. By treating questionnaires “as programs,” they simulated execution to find unreachable questions and unintended skips, using visualization (Nassi-Shneiderman diagrams [15]) to communicate complex logic to designers.

1.3.5 Limitations of Prior Approaches

While these approaches made significant contributions, they share common limitations:

- **Documentation-focused:** Tools like TADEQ primarily support visualization and manual validation
- **Simulation-based:** Path enumeration approaches (Feeney & Feeney) require exhaustive or sample-based traversal of respondent paths
- **Scalability constraints:** Complete path enumeration becomes infeasible for complex questionnaires
- **No formal guarantees:** Testing-based approaches cannot prove absence of problems, only detect present ones—a fundamental limitation of testing [16] that applies to all software systems, not just questionnaires

1.4 The Declarative Revolution: From Imperative to Functional Paradigms

Traditional questionnaire systems often employ an **imperative paradigm**, where:

- Questionnaire execution is a sequential process with mutable state.
- Each question modifies program variables that control subsequent flow.
- Skip logic is implemented through if-then-else statements in procedural code.
- Validation requires executing the questionnaire with test data.

This imperative approach, while intuitive for implementation, creates challenges for formal analysis. State mutations make it difficult to reason about all possible execution paths. Forward-only execution obscures the dependencies between questions. Testing can sample paths but cannot provide completeness guarantees.

1.4.1 Declarative Specification of Questionnaire Logic

Our approach adopts a **declarative, constraint-based paradigm** inspired by functional programming principles and influenced by Hoare Logic [17]. Just as Hoare triplets $\{P\}C\{Q\}$ specify program correctness through preconditions and postconditions, our framework specifies questionnaire correctness through logical constraints:

- **Preconditions as predicates:** Each question’s accessibility is specified by a Boolean formula over all answers, not sequential conditions—analogous to Hoare’s precondition P that must hold before execution
- **Postconditions as constraints:** Valid responses are characterized by mathematical constraints rather than procedural validation code—analogous to Hoare’s postcondition Q that must hold after execution
- **Immutable semantics:** Answer variables represent fixed facts about a complete questionnaire response (e.g., “age equals 25”), not values that change during execution—we analyze all possible complete answer sets rather than simulating step-by-step questionnaire execution

- **Declarative dependencies:** Relationships between questions are explicit in the logical structure, not hidden in execution order

This declarative, functional-style specification fundamentally changes how we analyze questionnaires: instead of simulating execution paths through mutable state (as in imperative approaches), we reason about relationships between immutable values. Preconditions become pure functions mapping answer sets to Boolean values. Postconditions become constraints in a constraint satisfaction problem. The entire questionnaire becomes a set of logical relationships that can be analyzed compositionally—each question’s properties can be verified independently and combined, much like pure functions in functional programming can be reasoned about in isolation and composed together.

1.5 Contributions

This work makes the following key contributions:

1. **Declarative Formal Framework:** We present a mathematical framework for representing questionnaire logic using preconditions and postconditions as logical formulas, enabling static analysis without questionnaire execution. This framework unifies concepts from prior work (Willenborg’s routing/edit structures [8], Fellegi-Holt’s consistency requirements [7], graph-based dependency representations [9, 11]) into a coherent formal system.
2. **SMT-Based Automated Verification:** We develop techniques for translating questionnaire logic into SMT constraints, enabling automated analysis using the Z3 theorem prover [18]. Unlike simulation-based approaches [14] or testing frameworks [11], our approach provides mathematical guarantees about questionnaire properties.
3. **Imperative-to-Declarative Compilation:** We design and implement a compiler that transforms embedded imperative Python code blocks into equivalent declarative Z3 constraints through Single Static Assignment (SSA) [19], conditional branching transformation, and bounded loop unrolling [20]. This bridges practical questionnaire authoring with formal verification.

4. **Complexity-Aware Practical Implementation:** Recognizing that questionnaire consistency checking is NP-complete [5], we demonstrate that modern SMT solvers can nevertheless handle real-world questionnaires effectively, providing actionable feedback to designers.

1.6 Thesis Organization

- **Chapter 2:** Surveys prior research in questionnaire validation and the formal methods technologies that enable our approach.
- **Chapter 3:** Provides formal definitions of questionnaires, including preconditions, postconditions, dependency graphs, and complex item types.
- **Chapter 4:** Details the compilation of questionnaire logic to Z3 constraints, including SSA transformation, loop unrolling, and complex type encoding.
- **Chapter 5:** Clarifies the relationship between global satisfiability and per-item validation with practical consequences for tool design.
- **Chapter 6:** Summarizes contributions, discusses limitations, and outlines future research directions.

Through this formal approach to questionnaire validation, we aim to improve the reliability and effectiveness of AI-generated surveys while honoring the decades of theoretical and practical work that preceded this thesis. By applying SMT solvers to problems that Willenborg proved NP-complete, we demonstrate that computational complexity results need not be barriers when armed with modern algorithmic techniques. By formalizing concepts that graph-based approaches visualized, we enable automated verification of properties that previous tools could only check manually.

The synthesis of historical questionnaire research with modern formal methods creates a path toward questionnaires that are not just well-designed from a psychometric perspective [21, 22], but also logically sound from a formal verification perspective—ensuring that complex questionnaire logic behaves as intended and provides meaningful, complete data collection experiences for all respondents.

Chapter 2

Related Work

This chapter surveys the research landscape that forms the foundation for our work, examining both the historical development of questionnaire validation methods and the formal methods technologies that enable our approach. We explore how satisfiability modulo theories (SMT) solvers [23, 18] have evolved to become practical tools for questionnaire verification, and position our contributions relative to prior work in graph-based validation, path enumeration, and edit rule consistency checking.

2.1 Satisfiability Modulo Theories: The Enabling Technology

The theoretical advances in questionnaire logic analysis (formalization, graph representations, complexity understanding) set the stage, but practical automated validation required computational breakthroughs. Satisfiability Modulo Theories (SMT) solvers provide the enabling technology.

Boolean Satisfiability (SAT) The Boolean Satisfiability problem [24] determines whether there exists an assignment of truth values to Boolean variables making a given formula true. SAT is historically significant as the first proven NP-complete problem [25], yet modern SAT solvers efficiently handle instances with millions of variables through sophisticated algorithmic advances developed over the past two decades.

While SAT provides the computational foundation, questionnaire validation requires reasoning beyond pure Boolean logic—we need to express constraints on integer answer values, arithmetic relationships, and structured data types. This motivates the extension to Satisfiability Modulo Theories.

From SAT to SMT: Adding Mathematical Theories While SAT solvers work with pure Boolean logic, questionnaires require reasoning about integers (answer values, ranges), arithmetic relationships, and complex data structures. Satisfiability Modulo Theories (SMT) extends SAT by combining Boolean satisfiability with decision procedures for various mathematical theories [24, 18]:

Linear Integer Arithmetic (LIA): Constraints involving integer variables, equations, and inequalities—essential for answer domains and numeric preconditions

Arrays: Theory of arrays with select and store operations—useful for representing vector and matrix question types

Uninterpreted Functions: Functions without concrete implementations, enabling abstraction

Theory Combination: Methods for handling formulas mixing multiple theories simultaneously

SMT Architecture and Application Modern SMT solvers integrate Boolean satisfiability checking with theory reasoning through a layered architecture known as DPLL(T)—an extension of the Davis-Putnam-Logemann-Loveland (DPLL) SAT solving algorithm parameterized by a background theory T [26]. In this framework, a SAT engine handles the Boolean structure of formulas, while specialized theory solvers check constraints in specific mathematical domains (arithmetic, arrays, etc.). When the SAT engine proposes a Boolean assignment that violates theory constraints, the theory solver provides an explanation, allowing the SAT engine to learn from the conflict and avoid similar invalid assignments in the future.

For questionnaire validation, SMT solvers excel at several critical tasks:

- **Reachability Analysis:** Determine whether each question can be reached under some valid answer sequence—exactly the problem Elliott [11] and others addressed through path enumeration, but with formal guarantees

- **Consistency Checking:** Verify that routing conditions and edit constraints are not contradictory—solving Fellegi and Holt’s [7] edit consistency problem at the design stage
- **Counterexample Generation:** When problems exist, generate concrete answer sequences demonstrating the issue—providing actionable diagnostic information for designers
- **Completeness Verification:** Prove that all intended questionnaire behaviors are possible—going beyond testing to mathematical proof

The Z3 Theorem Prover Z3 [18], developed by Microsoft Research, exemplifies modern SMT solvers. It provides:

- **Rich API:** Interfaces for Python, C++, Java, and .NET
- **Extensive Theory Support:** Arithmetic, bit-vectors, arrays, strings, and algebraic datatypes
- **Model Generation:** Concrete variable assignments when formulas are satisfiable

The Z3 Python API enables intuitive constraint specification, making SMT solving accessible to questionnaire designers and researchers without deep formal methods expertise.

2.2 Bridging Questionnaire Validation and SMT Solving

This thesis synthesizes six decades of questionnaire logic research with modern SMT solving technology. Where Willenborg formalized routing and edit structures, we provide automated verification. Where Bethlehem, Elliott, and Feeney enumerated paths for testing, we prove reachability through constraint solving. Where traditional approaches documented and detected problems, we provide formal guarantees about questionnaire correctness.

Our key insight is that questionnaire validation, when specified declaratively, maps naturally to SMT constraint satisfaction. By treating:

- **Routing conditions** as precondition formulas (building on Willenborg’s terminology)
- **Edit rules** as postcondition constraints (building on Fellegi-Holt’s framework)
- **Question dependencies** as a dependency graph (building on graph-based approaches)
- **Questionnaire validity** as a global satisfiability problem (formalized in Chapter 5)

Comparison with Graph-Based Approaches Graph-based methods [10, 11, 12] provide excellent visualization and documentation of questionnaire logic. TADEQ [10] represents questionnaires as directed graphs with single start/end nodes. Elliott [11] derives basis path sets for systematic testing. Our work complements these approaches:

- **Shared foundation:** We use dependency graphs for cycle detection and coverage analysis
- **Enhanced analysis:** SMT solving enables formal verification beyond what graph visualization provides
- **Diagnostic precision:** Graph visualizations annotated with reachability properties pinpoint exact locations of logical inconsistencies

Comparison with Path Enumeration Feeney and Feeney [14] enumerate all questionnaire paths through simulation, generating concrete scenarios for validation. Our approach differs fundamentally:

- **Exhaustive vs. Existence:** Path enumeration must explore all possibilities; SMT solving proves existence or absence of paths through constraint satisfaction
- **Scalability:** For questionnaires with n questions and average branching factor b , path enumeration may require exploring $O(b^n)$ paths; SMT solvers use constraint propagation and learning to prune the search space drastically [26]
- **Completeness:** Our approach provides completeness guarantees (all questions reachable) without enumerating all paths

Extending Edit Rule Consistency Fellegi and Holt [7] established that edit rules must admit at least one consistent response. Our postcondition validity checking extends this principle from post-collection data cleaning to pre-deployment questionnaire design. We verify not just that global consistency is possible, but that every reachable question has satisfiable postconditions given its preconditions.

2.3 Formal Methods in Software Verification

The application of formal methods to questionnaire validation leverages decades of research in software verification. SMT solvers have become indispensable in verifying program correctness [27], finding software bugs through symbolic execution [28, 29], and bounded model checking [20].

Symbolic Execution and Questionnaire Analysis Symbolic execution [28, 29] explores program paths using symbolic rather than concrete values. SMT solvers determine path feasibility, generate inputs triggering specific behaviors, and detect assertion failures. Questionnaire validation exhibits striking parallels:

- **Path feasibility** $\leftrightarrow$ Question reachability
- **Input generation** $\leftrightarrow$ Witness answer sequences
- **Assertion failures** $\leftrightarrow$ Postcondition violations

By viewing questionnaires as programs where answer variables are symbolic inputs and preconditions are conditional branches, we can apply symbolic execution techniques to questionnaire validation.

Bounded Model Checking Bounded model checking [30, 20] verifies systems up to a certain depth, systematically exploring state spaces within bounded limits. When questionnaires contain loops (through code blocks or iterative question structures), we apply bounded analysis: loop unrolling for statically determinable bounds enables analysis of iterative logic while maintaining decidability.

Chapter 3

Questionnaire Definition

This chapter establishes the mathematical framework for formally representing questionnaires as logical systems. We build up the formal definitions from axiomatic components to complete questionnaire structures, then analyze their properties and validation criteria.

3.1 Axiomatic Building Blocks

We begin by defining the fundamental components that constitute a questionnaire. These building blocks will be composed to form the complete questionnaire structure.

Definition 1 (Item). An **item** I_i is a questionnaire element that presents content to respondents and optionally collects responses. Items may take several forms:

- **Informational items:** Comments or instructions without associated outcomes (no answer variable)
- **Scalar questions:** Single questions with a single integer outcome
- **Question groups:** Collections of related sub-questions sharing common logical properties
- **Matrix questions:** Grid-based questions where all cells share a common domain (e.g., rating scales)

Items form the basic structural units of a questionnaire, indexed as $I_1, I_2, \dots, I_n$ for a questionnaire with n items.

Definition 2 (Outcome Variable). For each item I_i that collects responses, there is an associated **outcome variable** S_i representing the respondent's answer. The type of S_i depends on the item type:

- For scalar questions: $S_i \in \mathbb{Z}$ (single integer)
- For question groups: $S_i \in \mathbb{Z}^k$ (vector of k integers)
- For matrix questions: $S_i \in \mathbb{Z}^{m \times n}$ (matrix of $m \times n$ integers)
- For informational items: no associated outcome variable

Assignment Let $\mathbf{S} := (S_1, \dots, S_n)$ denote the vector of outcome variables. An *assignment* is a vector $\mathbf{a} := (a_1, \dots, a_n)$ over the corresponding domains. We write $\mathbf{a} \models \phi$ to denote that the formula ϕ evaluates to true under the assignment $\mathbf{a}$.

We denote the complete vector of outcome variables as $\mathbf{S} = (S_1, \dots, S_n)$. For simplicity, this chapter and Chapter 5 focus on scalar items. Section 3.7 extends the framework to vector and matrix types.

Definition 3 (Domain Constraint). Each outcome variable S_i has an associated **domain constraint** $D_i(S_i)$ that restricts its valid values. For scalar outcomes, the domain can be specified in two ways:

- **Range constraint:** $D_i(S_i) : \ell_i \leq S_i \leq u_i$, where ℓ_i and u_i are the lower and upper bounds respectively
- **Enumeration constraint:** $D_i(S_i) : S_i \in \{v_1, v_2, \dots, v_k\}$, where $\{v_1, v_2, \dots, v_k\}$ is a finite set of allowed integer values

The conjunction of all domain constraints forms the base constraint:

$$B := \bigwedge_{i=1}^n D_i(S_i)$$

Let the set of domain-respecting assignments be:

$$\text{Dom} := \left\{ \mathbf{a} \mid \bigwedge_{i=1}^n D_i(a_i) \text{ holds} \right\}$$

Definition 4 (Precondition). Each item I_i has a **precondition** P_i , a Boolean formula over the outcome variables $\mathbf{S} = (S_1, \dots, S_n)$ that determines whether the item is presented to the respondent.

- If an assignment $\mathbf{a}$ satisfies P_i (written $\mathbf{a} \models P_i$), then item I_i is accessible (displayed)
- If $\mathbf{a} \not\models P_i$, then item I_i is not presented

Preconditions encode the conditional logic that creates branching paths through the questionnaire based on previous answers.

Definition 5 (Postcondition). Each item I_i has a **postcondition** Q_i , a Boolean formula over the outcome variables $\mathbf{S} = (S_1, \dots, S_n)$ that constrains valid responses for that item.

- Postconditions are only evaluated when the item is accessible (i.e., when $\mathbf{a} \models P_i$ for the current assignment $\mathbf{a}$)
- Valid questionnaire responses must satisfy: $P_i \implies Q_i$ (if the precondition holds, then the postcondition must hold)

Postconditions ensure logical consistency between answers and can dynamically restrict acceptable values based on prior responses.

3.2 Questionnaire Structure

Having defined the building blocks, we now formally define a complete questionnaire structure.

Definition 6 (Questionnaire). A **questionnaire** $\mathcal{G}$ is a tuple $(\mathcal{I}, \mathbf{S}, \mathcal{D}, \mathcal{P}, \mathcal{Q}, I_{\text{start}})$ where:

- $\mathcal{I} = \{I_1, \dots, I_n\}$ is a finite set of items
- $\mathbf{S} = (S_1, \dots, S_n)$ is a vector of outcome variables
- $\mathcal{D} = (D_1, \dots, D_n)$ where D_i specifies the domain constraint for S_i
- $\mathcal{P} = (P_1, \dots, P_n)$ where P_i is the precondition formula for I_i over variables $\mathbf{S}$

- $\mathcal{Q} = (Q_1, \dots, Q_n)$ where Q_i is the postcondition formula for I_i over variables $\mathbf{S}$
- $I_{\text{start}} \in \mathcal{I}$ is the designated starting item

Satisfiability Notation:

- $\text{SAT}(\phi)$ indicates ϕ has a model,
i.e. there is an assignment which makes ϕ true.
- $\text{UNSAT}(\phi)$ indicates no model exists,
i.e. there is no assignment which makes ϕ true.
- $\Gamma \models \phi$ denotes semantic entailment.

Remark on Formal Logic Notation. The symbol “ $\models$ ” is used in two related senses following standard logical notation:

- $\mathbf{a} \models \phi$ means assignment $\mathbf{a}$ (a valuation over the outcome variables) *satisfies* formula ϕ (the formula evaluates to true under that assignment)
- $\Gamma \models \phi$ means formula set Γ *entails* formula ϕ (every model of Γ is also a model of ϕ)

Context determines which sense applies:

- if the left side is an assignment (valuation), it denotes satisfaction;
- if the left side is a formula or set of formulas, it denotes entailment.

Example: Entailment in Questionnaire Analysis. Consider a health survey with age-restricted questions about alcohol consumption:

- I_1 : “What is your age?” with $S_1 \in [1, 120]$
- I_2 : “Do you consume alcohol?” with $S_2 \in \{0, 1\}$ (0=No, 1=Yes)
- I_3 : “How many drinks per week?” with $S_3 \in [0, 100]$

With constraints:

- $P_2 = (S_1 \geq 18)$ — alcohol question only asked if age ≥ 18

- $P_3 = (S_1 \geq 18) \wedge (S_2 = 1)$ — drinks question only if legal age AND consumes alcohol
- $Q_3 = (S_3 > 0)$ — if answering drinks question, must report at least 1 drink

Let $\Gamma = \{S_1 \geq 18, S_2 = 1, P_3, Q_3\}$ represent “person is 18+, consumes alcohol, item 3 is reached, and its postcondition holds.”

Let $\phi = (S_3 > 0)$ be “person drinks at least 1 drink per week.”

Then $\Gamma \models \phi$ because every assignment satisfying all formulas in Γ must also satisfy ϕ , since Q_3 is defined as $(S_3 > 0)$. This entailment helps detect logical redundancies and validate questionnaire consistency.

3.3 Precondition Properties

We now analyze the properties of preconditions, specifically their reachability characteristics.

Item Reachability: For each item I_i , we classify its precondition P_i with respect to the base constraints B into three categories:

$$\begin{aligned} \text{ALWAYS} : & \quad \text{UNSAT}(B \wedge \neg P_i) \\ \text{NEVER} : & \quad \text{UNSAT}(B \wedge P_i) \\ \text{CONDITIONAL} : & \quad \text{otherwise (i.e., } \equiv \text{SAT}(B)) \end{aligned}$$

Always Reachable

P_i is **always reachable** (always satisfiable) if for every valid assignment $\mathbf{a}$ (i.e., $\mathbf{a}$ satisfies the domain constraints), $\mathbf{a}$ satisfies P_i .

Formally:

$$\forall \mathbf{a} \in \text{Dom}, \quad \mathbf{a} \models P_i$$

where Dom is the set of all domain-respecting assignments. In practice, this means item I_i is always presented regardless of how previous questions are answered.

Never Reachable

P_i is **never reachable** (never satisfiable) if for every valid assignment $\mathbf{a}$, $\mathbf{a}$ does not satisfy P_i .

Formally:

$$\forall \mathbf{a} \in \text{Dom}, \quad \mathbf{a} \not\models P_i$$

This indicates a logical error—there is no scenario where I_i can be accessed.

Conditionally Reachable

P_i is **conditionally reachable** (conditionally satisfiable) if neither always nor never reachable; equivalently, there exists at least one valid assignment where P_i is satisfied and at least one where it is not satisfied.

Formally, there exist two domain-respecting assignments such that:

$$\begin{aligned} \exists \mathbf{a} \in \text{Dom}, \quad \mathbf{a} &\models P_i \\ \exists \mathbf{b} \in \text{Dom}, \quad \mathbf{b} &\not\models P_i \end{aligned}$$

This is the normal case for conditional questions that appear based on prior answers.

Algorithm 1 Precondition Reachability Classification

Require: Base constraints B , precondition P_i

Ensure: Reachability status: ALWAYS, NEVER, or CONDITIONAL

```

1: Check SAT( $B \wedge \neg P_i$ ) using SMT solver
2: if result is UNSAT then
3:   return ALWAYS                                ▷ Item is always reachable
4: end if
5: Check SAT( $B \wedge P_i$ ) using SMT solver
6: if result is UNSAT then
7:   return NEVER                                ▷ Item is never reachable
8: else
9:   return CONDITIONAL                          ▷ Item is conditionally reachable
10: end if

```

3.4 Postcondition Properties

We analyze postconditions relative to their associated preconditions, as postconditions only matter when items are reached.

Item Invariant Classification: Let $B := \bigwedge_{k=1}^n D_k(S_k)$ denote the base constraints, P_i the precondition, and Q_i the postcondition of item I_i . We classify postconditions relative to their preconditions:

VACUOUS :	$\text{UNSAT}(B \wedge P_i)$
TAUTOLOGICAL :	$\text{UNSAT}(B \wedge P_i \wedge \neg Q_i)$
CONSTRAINING :	$\text{SAT}(B \wedge P_i \wedge Q_i)$ and $\text{SAT}(B \wedge P_i \wedge \neg Q_i)$
INFEASIBLE :	$\text{UNSAT}(B \wedge P_i \wedge Q_i)$

Vacuous Postconditions

The item is never reachable (precondition is never satisfiable), so the postcondition check is **vacuous**—it has no practical significance since the item never appears.

Formally:

$$\forall \mathbf{a} \in \text{Dom}, \quad \mathbf{a} \not\models P_i$$

Tautological Postconditions

Q_i is **tautological** relative to P_i if whenever the item is reached (under valid domain assignments), the postcondition is automatically satisfied regardless of the answer given.

Formally:

$$\forall \mathbf{a} \in \text{Dom}, \quad (\mathbf{a} \models P_i) \implies (\mathbf{a} \models Q_i)$$

This typically indicates the postcondition adds no meaningful constraint beyond what the precondition and domain constraints already enforce.

Constraining Postconditions

Q_i is **constraining** if it meaningfully restricts the acceptable answers when the item is reached. There exist valid assignments where the item is reached and the postcondition is satisfied, as well as assignments where it is reached but the postcondition is violated.

Formally, there exist two domain-respecting assignments such that:

$$\begin{aligned} \exists \mathbf{a} \in \text{Dom}, \quad & (\mathbf{a} \models P_i) \text{ and } (\mathbf{a} \models Q_i) \\ \exists \mathbf{b} \in \text{Dom}, \quad & (\mathbf{b} \models P_i) \text{ and } (\mathbf{b} \not\models Q_i) \end{aligned}$$

This is the normal case where postconditions provide meaningful validation of responses.

Infeasible Postconditions

Q_i is **infeasible** if it cannot be satisfied whenever the item is reached under valid domain assignments.

Formally:

$$\forall \mathbf{a} \in \text{Dom}, \quad (\mathbf{a} \models P_i) \implies (\mathbf{a} \not\models Q_i)$$

Meaning that there does not exist any $\mathbf{a} \in \text{Dom}$ such that both $\mathbf{a} \models P_i$ and $\mathbf{a} \models Q_i$. This represents a logical error—the item asks for an impossible answer given its preconditions and domain constraints.

Algorithm 2 Postcondition Classification

Require: Base constraints B , precondition P_i , postcondition Q_i

Ensure: Classification: VACUOUS, TAUTOLOGICAL, CONSTRAINING, or INFEASIBLE

```

1: Check SAT( $B \wedge P_i$ ) using SMT solver
2: if result is UNSAT then
3:   return VACUOUS ▷ Item is never reachable
4: end if
5: Check SAT( $B \wedge P_i \wedge Q_i$ ) using SMT solver
6: if result is UNSAT then
7:   return INFEASIBLE ▷ Postcondition cannot be satisfied when reached
8: end if
9: Check SAT( $B \wedge P_i \wedge \neg Q_i$ ) using SMT solver
10: if result is UNSAT then
11:   return TAUTOLOGICAL ▷ Postcondition always holds when reached
12: else
13:   return CONSTRAINING ▷ Postcondition meaningfully constrains
14: end if

```

Global satisfiability of Q_i . For completeness, we may also record the global satisfiability properties of Q_i independent of P_i :

$$q_globally_false := \text{UNSAT}(B \wedge Q_i), \quad q_globally_true := \text{UNSAT}(B \wedge \neg Q_i)$$

These do not alter the precondition-based reachability classes but can be reported as diagnostic properties.

3.5 Implementation via Python

In practice, the mathematical formulas for preconditions and postconditions are implemented using Python expressions and statements that are compiled to formal constraints.

Python Expressions Preconditions P_i and postconditions Q_i are specified as lists of Python *boolean expressions*. These expressions:

- Reference outcome variables through item identifiers or intermediate variables from code blocks
- Support arithmetic, comparison, and logical operators
- Return boolean values for constraint evaluation

Python Statements While preconditions and postconditions are expressed as boolean expressions, complex questionnaire logic often requires intermediate computations that would make these expressions unwieldy or repetitive. Code blocks address this by allowing the introduction of auxiliary variables that store computed values derived from previous items' outcomes. For example, rather than repeating a complex calculation like $(S_1 + S_2) * 0.15 + S_3$ in multiple preconditions, one can compute `total_cost = (S_1 + S_2) * 0.15 + S_3` once in a code block and reference `total_cost` in subsequent conditions.

Items may contain Python *code blocks* for these complex computations. These blocks:

- Execute after the answer variable is assigned
- Can perform intermediate calculations and variable assignments
- Support control flow (if-statements, for-loops with bounded iteration)

Compilation to Z3 Constraints. Both expressions and statements are compiled to Z3 SMT solver constraints through:

- Static analysis of expression ASTs for boolean expressions
- SSA transformation for variable assignments in statement blocks

- Bounded loop unrolling for iterations
- Conditional branching transformation for if-statements

Details of the compilation process are covered in Chapter 4. Throughout this thesis, we use the term *Boolean formula* in the mathematical sense to refer abstractly to these predicates, while the concrete implementation uses Python expressions that are compiled to first-order logic constraints for solver-based analysis.

3.6 Dependency Graph and Cycle Test

To ensure questionnaires can be evaluated in a consistent order, we analyze dependencies between items.

Definition 7 (Path in Dependency Graph). A **path** from item I_j to item I_i in the dependency graph $G = (V, E)$ is a sequence of distinct items

$$\pi = (I_j = I_{k_0}, I_{k_1}, \dots, I_{k_m} = I_i)$$

where $m \geq 0$ and for each $\ell \in \{0, \dots, m-1\}$, there exists an edge $(k_\ell \rightarrow k_{\ell+1}) \in E$.

When $m = 0$, the path consists of a single item $(I_j = I_i)$, representing a trivial path from an item to itself.

Dependency Graph Construction The dependency graph $G = (V, E)$ captures data flow dependencies between items through both direct variable usage and intermediate computations in code blocks. Construction proceeds in three stages.

Stage 1: Build data flow graph. Through static analysis of preconditions, postconditions, and code blocks, we construct an intermediate directed graph where vertices represent both items and variables:

- **Item vertices:** Each questionnaire item I_i is a vertex
- **Variable vertices:** Each variable (item outcome S_i or intermediate variable v) is a vertex

We extract dependency edges by analyzing all code blocks and formulas:

- When item I_j is answered, its outcome S_j is assigned $\Rightarrow$ edge $(I_j \rightarrow S_j)$

- When code assigns item outcome to variable v (e.g., $v = S_j$) $\Rightarrow$ edge $(S_j \rightarrow v)$
- When code modifies variable v based on S_j (e.g., $v += S_j$) $\Rightarrow$ edge $(S_j \rightarrow v)$
- When variable v is used in I_i 's precondition P_i or postcondition $Q_i \Rightarrow$ edge $(v \rightarrow I_i)$
- When S_j is directly used in P_i or $Q_i \Rightarrow$ edge $(S_j \rightarrow I_i)$

Example: Data Flow Graph Construction. Consider a survey with intermediate calculations:

- I_1 : “Your monthly rent” with outcome S_1
- I_2 : “Your monthly utilities” with outcome S_2
- Code block: `total_housing = S_1 + S_2`
- I_3 : “Other monthly expenses” with outcome S_3 , precondition $P_3 = (\text{total_housing} < 3000)$

Data flow edges:

- $(I_1 \rightarrow S_1)$ — answering I_1 assigns S_1
- $(I_2 \rightarrow S_2)$ — answering I_2 assigns S_2
- $(S_1 \rightarrow \text{total_housing})$ — S_1 used to compute `total_housing`
- $(S_2 \rightarrow \text{total_housing})$ — S_2 used to compute `total_housing`
- $(\text{total_housing} \rightarrow I_3)$ — `total_housing` used in P_3

Stage 2: Compute item-to-item dependencies. From the data flow graph, we construct the item dependency graph $G = (V, E)$ where $V = \{I_1, \dots, I_n\}$ contains only item vertices. An edge $(I_j \rightarrow I_i) \in E$ exists if and only if there is a path in the data flow graph from I_j to I_i through any sequence of variable vertices.

This captures transitive dependencies through intermediate variables: if I_j 's outcome flows through variables $v_1, \dots, v_k$ to affect I_i 's evaluation, then I_i depends on I_j .

Example: Item Dependency Extraction. Continuing the previous example, the data flow graph contains paths:

$$I_1 \rightarrow S_1 \rightarrow \text{total_housing} \rightarrow I_3$$

$$I_2 \rightarrow S_2 \rightarrow \text{total_housing} \rightarrow I_3$$

Resulting item dependencies:

- $(I_1 \rightarrow I_3)$ — I_3 depends on I_1
- $(I_2 \rightarrow I_3)$ — I_3 depends on I_2

Item I_3 cannot be evaluated until both I_1 and I_2 have been answered.

General example. Consider code that introduces variable v , assigns $v = S_j$, then updates v based on S_k (e.g., $v = v + S_k$), and finally uses v in both P_i and P_m . The data flow graph contains:

$$I_j \rightarrow S_j \rightarrow v, \quad I_k \rightarrow S_k \rightarrow v, \quad v \rightarrow I_i, \quad v \rightarrow I_m$$

resulting in item dependencies: $(I_j \rightarrow I_i)$, $(I_j \rightarrow I_m)$, $(I_k \rightarrow I_i)$, $(I_k \rightarrow I_m)$.

Stage 3: Topological ordering. To determine a valid evaluation order, we compute a topological sort of G using Kahn's algorithm [31]:

Algorithm 3 Kahn's Algorithm for Topological Sorting

```

1: Initialize:  $S \leftarrow \{\text{items with in-degree } 0\}$ ,  $L \leftarrow \text{empty list}$ 
2: while  $S$  is non-empty do
3:   Remove an item  $n$  from  $S$ 
4:   Add  $n$  to tail of  $L$ 
5:   for each item  $m$  with edge  $(n \rightarrow m)$  do
6:     Remove edge  $(n \rightarrow m)$  from graph
7:     if  $m$  has in-degree 0 then
8:       Add  $m$  to  $S$ 
9:     end if
10:  end for
11: end while
12: if graph still has edges then
13:   report cycle detected
14: else
15:   return  $L$  as valid topological order
16: end if
```

The resulting topological order L ensures that items are evaluated in dependency-respecting sequence: when item I_i is reached, all variables referenced by P_i have been assigned.

Example: Topological Ordering. Consider a survey with dependencies:

- I_1 : “Annual income” (no dependencies, in-degree 0)
- I_2 : “Monthly savings” (no dependencies, in-degree 0)
- I_3 : “Savings rate” with P_3 depending on S_1 and S_2 (in-degree 2)
- I_4 : “Financial health” with P_4 depending on S_3 (in-degree 1)

Dependency graph: $(I_1 \rightarrow I_3), (I_2 \rightarrow I_3), (I_3 \rightarrow I_4)$

Kahn’s algorithm execution:

1. Initially $S = \{I_1, I_2\}$ (in-degree 0)
2. Remove I_1 , add to $L = [I_1]$, decrement in-degree of I_3 to 1
3. Remove I_2 , add to $L = [I_1, I_2]$, decrement in-degree of I_3 to 0
4. Add I_3 to S , remove I_3 , add to $L = [I_1, I_2, I_3]$, decrement in-degree of I_4 to 0
5. Add I_4 to S , remove I_4 , add to $L = [I_1, I_2, I_3, I_4]$
6. No edges remain, return L

Valid evaluation order: Items can be evaluated in the order I_1, I_2, I_3, I_4 (or I_2, I_1, I_3, I_4).

Cycle Test A questionnaire has **no cycles** if G is a **Directed Acyclic Graph (DAG)**. Equivalently, there is no sequence of distinct items $i_1, i_2, \dots, i_k$ ($k \geq 2$) such that $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_k \rightarrow i_1$.

Kahn’s algorithm [31] detects cycles: if the algorithm terminates with edges remaining in the graph, those edges form cycles. Cycles threaten on-the-fly navigation because no item on the cycle can be evaluated first—each depends on others in the cycle.

Example: Cycle Detection. Consider a survey with circular dependency (design error):

- I_1 : “Estimated total cost” with precondition P_1 depending on S_3
- I_2 : “Budget available” with precondition P_2 depending on S_1
- I_3 : “Cost overrun” with precondition P_3 depending on S_2

Dependency graph: $(I_1 \rightarrow I_2), (I_2 \rightarrow I_3), (I_3 \rightarrow I_1)$

Kahn’s algorithm execution:

1. Initially $S = \{\}$ (all items have in-degree 1, none can start)
2. Algorithm terminates immediately with edges remaining
3. Return **CYCLE DETECTED**: $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow I_1$

This circular dependency makes evaluation impossible—no item can be evaluated first because each depends on another item in the cycle.

3.7 Extension to Complex Item Types

While the core concepts of questionnaire validation have been presented using single integer answer variables for clarity, real-world questionnaires often require more complex answer structures. This section extends our formal framework to handle vector and matrix item types.

Motivation for Complex Types Modern questionnaires frequently employ question formats that cannot be adequately represented by single integer values:

- **Ranking Questions:** “Please rank these 5 features by importance (1 = most important, 5 = least important)” requires a vector of integers where each element appears exactly once.
- **Multiple Selection Questions:** “Select all symptoms you have experienced” produces a binary vector indicating which options were selected.
- **Allocation Questions:** “Distribute 100 points among these 4 categories to indicate their relative importance” requires a vector with a sum constraint.

- **Grid Questions:** “For each product, rate the following attributes on a scale of 1-10” generates a matrix where rows represent products and columns represent attributes.
- **Correlation Matrices:** “Indicate the relationship strength between each pair of factors” creates a symmetric matrix of values.

These question types are essential for capturing nuanced respondent preferences, multi-dimensional assessments, and complex relationships that single-value questions cannot express. The Questionnaire Markup Language (QML) [32] provides native support for these complex types through `QuestionGroup` and `MatrixQuestion` constructs.

Vector and Matrix Item Types We extend our framework to support three item types, where each item I_i has an associated answer variable S_i of one of the following types:

Scalar Type: $S_i \in \mathbb{Z}$

A single integer value with domain constraint $D_i : \ell_i \leq S_i \leq u_i$

Vector Type: $\mathbf{S}_i \in \mathbb{Z}^{k_i}$

A vector of k_i integers with domain constraint:

$$D_i : \bigwedge_{j=1}^{k_i} (\ell_{i,j} \leq S_{i,j} \leq u_{i,j})$$

Matrix Type: $\mathbf{S}_i \in \mathbb{Z}^{m_i \times n_i}$

A matrix of $m_i \times n_i$ integers with domain constraint:

$$D_i : \bigwedge_{j=1}^{m_i} \bigwedge_{k=1}^{n_i} (\ell_{i,j,k} \leq S_{i,j,k} \leq u_{i,j,k})$$

Unified Notation To maintain notational consistency across all item types, we adopt the following conventions:

- S_i denotes the answer variable for item I_i regardless of its type
- For scalar items: S_i directly represents the integer value

- For vector items: S_i represents the entire vector, with $S_{i,j}$ or $S_i[j]$ denoting the j -th component
- For matrix items: S_i represents the entire matrix, with $S_{i,j,k}$ or $S_i[j][k]$ denoting the element at row j , column k

This notation allows us to write preconditions and postconditions uniformly while accessing components when needed.

Extended Domain Constraints The base constraint formula naturally extends to accommodate complex types:

Definition 8 (Base Constraint for Complex Types). The *base constraint* B for a questionnaire with n items of potentially complex types is defined as:

$$B := \bigwedge_{i=1}^n D_i(S_i)$$

where $D_i(S_i)$ denotes the domain constraint for item i , defined according to the type of S_i :

For each type, $D_i(S_i)$ is defined as follows:

- For scalar S_i : $D_i(S_i) := \ell_i \leq S_i \leq u_i$
- For vector $\mathbf{S}_i$: $D_i(\mathbf{S}_i) := \bigwedge_{j=1}^{k_i} (\ell_{i,j} \leq S_{i,j} \leq u_{i,j})$
- For matrix $\mathbf{S}_i$: $D_i(\mathbf{S}_i) := \bigwedge_{j=1}^{m_i} \bigwedge_{k=1}^{n_i} (\ell_{i,j,k} \leq S_{i,j,k} \leq u_{i,j,k})$

3.7.1 Preconditions and Postconditions with Complex Types

Component Access in Conditions Preconditions and postconditions can reference individual components of vector and matrix items, enabling sophisticated conditional logic:

- **Vector component access:** P_i may include expressions like $(S_j[2] > 3)$ to check if the second element of vector item I_j exceeds 3
- **Matrix element access:** P_i may include $(S_j[1][3] = S_k[2][3])$ to compare specific matrix elements
- **Aggregate operations:** Conditions can use sums, products, or other aggregate functions over vector/matrix components

3.7.2 Common Constraint Patterns

Ranking Constraints For a ranking question with k options, the postcondition typically ensures:

1. All values are within range $[1, k]$
2. All values are distinct (each rank used exactly once)

Formally, for ranking item I_i with k_i options:

$$Q_i := \left(\bigwedge_{j=1}^{k_i} 1 \leq S_{i,j} \leq k_i \right) \wedge \text{Distinct}(S_{i,1}, \dots, S_{i,k_i})$$

Allocation Constraints For allocation questions where respondents distribute a fixed total across categories:

$$Q_i := \left(\sum_{j=1}^{k_i} S_{i,j} = T_i \right) \wedge \left(\bigwedge_{j=1}^{k_i} S_{i,j} \geq 0 \right)$$

where T_i is the total to be allocated (e.g., 100 points).

Matrix Symmetry Constraints For correlation or relationship matrices that must be symmetric:

$$Q_i := \bigwedge_{j=1}^{m_i} \bigwedge_{k=j+1}^{n_i} (S_{i,j,k} = S_{i,k,j})$$

Complete QML examples demonstrating vector and matrix item types in realistic survey scenarios can be found in Appendix A.

Chapter 4

Questionnaire Compilation

Electronic questionnaires combine two distinct paradigms: **declarative specifications** that define logical relationships between questions, and **imperative code blocks** that perform computations and state transformations. This hybrid structure allows questionnaire designers to express both mathematical constraints and procedural logic naturally, but it presents a fundamental challenge for formal verification.

A questionnaire consists of both declarative and imperative units that work together to define the survey logic:

Declarative Units (Python Expressions): These components specify the logical structure without describing how to compute results:

- **Domain constraints:** Define valid ranges for answer variables (e.g., `1 <= age <= 120`)
- **Preconditions:** Boolean expressions determining when questions appear (e.g., `income > 50000 and employed == True`)
- **Postconditions:** Validation rules that answers must satisfy (e.g., `total_expenses <= income`)

Imperative Units (Python Statements): These components perform computations and modify state through sequential execution:

- **Global initialization code:** Executed once at questionnaire start to set up shared variables and initial state

- **Item-level code blocks:** Executed after an item is answered and its post-condition is satisfied, typically to compute derived values or update running totals

For example, a financial survey might use declarative preconditions to control question flow based on income level, while using imperative code blocks to calculate tax brackets, accumulate expenses, or determine eligibility scores through complex multi-step algorithms.

The Translation Challenge To enable static analysis of questionnaire logic through SMT solving, we must translate both paradigms into a unified constraint representation. While declarative expressions map naturally to Z3 formulas, imperative code blocks require sophisticated compilation techniques:

- **Variable mutation:** Python allows reassignment (e.g., `total = total + expense`), but Z3 variables are immutable
- **Control flow:** If-statements create divergent execution paths that must be unified into single constraint expressions
- **Iteration:** Loops with bounded iterations must be unrolled into finite constraint sequences
- **Side effects:** Code blocks can modify multiple variables, requiring careful tracking of data dependencies

This chapter presents our compilation approach that transforms Python code—both expressions and statements—into equivalent Z3 constraints. The translation preserves the semantic behavior while enabling formal verification of questionnaire properties.

Compilation Techniques Overview Our Python-to-Z3 compiler employs three key techniques that handle the fundamental control flow constructs of imperative programming:

1. **Single Static Assignment (SSA):** Transforms variable mutations into versioned variables, where each assignment creates a new immutable variable

2. **Conditional Branching Transformation:** Converts if-statements into phi-functions that select values based on conditions
3. **Loop Unrolling:** Expands bounded loops into sequential constraints for finite iteration counts

The compilation process operates on Python Abstract Syntax Trees (AST), systematically transforming each node into equivalent Z3 constraints while maintaining semantic correctness for questionnaire validation purposes.

Pragmatic Design Philosophy The Z3 compiler follows a **pragmatic approach** that prioritizes questionnaire validation over perfect Python code validation:

- **Questionnaire-First Validation:** The primary goal is to ensure questionnaire flow logic is sound and all paths are reachable
- **Best-Effort Python Translation:** Python code blocks are translated to Z3 constraints using a “best-effort” approach
- **Graceful Degradation:** When Python constructs cannot be perfectly represented in Z3, the compiler continues with partial constraints rather than failing
- **Practical Static Analysis:** Focus on detecting unreachable questionnaire items and logical inconsistencies rather than comprehensive Python validation
- **Domain-Specific Optimization:** Optimized for survey logic patterns (conditionals, loops, arithmetic) rather than general Python semantics

This pragmatic approach ensures that questionnaire authors can validate their survey logic effectively while maintaining development velocity, even when dealing with complex Python expressions that may not have perfect Z3 equivalents.

Selective Compilation of Code Blocks During compilation to Z3 constraints, only the portions of code blocks that ultimately influence a precondition or postcondition are considered. Variables and computations that are never referenced in any condition are ignored, ensuring that the formal analysis focuses solely on logic that

affects questionnaire flow and validation. This selective approach reduces constraint complexity and allows questionnaire authors to include auxiliary computations (e.g., logging, debugging variables) without impacting validation performance.

4.1 Single Static Assignment (SSA)

Single Static Assignment is a fundamental technique in compiler design [19, 33] where each variable is assigned exactly once. When a variable is reassigned in the original program, we create a new versioned variable in the SSA form.

Motivation Consider the following Python code block within a questionnaire:

```
total = 0
total = total + item1
total = total + item2
result = total * tax_rate
```

Code 4.1: Python code with variable reassignment

In standard Python semantics, the variable `total` is mutated multiple times. However, Z3 variables are immutable once declared. SSA transformation resolves this by creating versioned variables.

SSA Transformation The SSA transformation creates a new variable for each assignment:

```
total_0 = 0
total_1 = total_0 + item1
total_2 = total_1 + item2
result_0 = total_2 * tax_rate
```

Code 4.2: SSA representation

Each assignment creates a new version of the variable, making the data flow explicit. This transforms mutable variables into immutable SSA variables that Z3 can directly represent.

In Z3, this SSA form translates directly to constraint variables:

```

total0 = IntVal(0)
total1 = total0 + Int('item1')
total2 = total1 + Int('item2')
result0 = total2 * Int('tax_rate')

```

Each variable becomes an immutable Z3 expression, eliminating the need for mutation and aligning perfectly with Z3’s functional constraint model.

4.2 Conditional Branching Transformation

Conditional statements in Python must be transformed into Z3’s functional representation using **If** expressions and constraint implications. When control flow paths merge after conditional statements, we need a mechanism to select which version of a variable should be used. This is accomplished through **phi functions** (written as ϕ), a fundamental concept in SSA form that is used specifically at control flow merge points.

Phi Functions: Merging Control Flow Paths A phi function ϕ is a special operation that selects between different versions of a variable based on which control flow path was taken. The general form is:

$$x_{\text{new}} = \phi(\text{condition}, x_{\text{true_branch}}, x_{\text{false_branch}})$$

This reads as: “If the condition is true, use the value from the true branch; otherwise, use the value from the false branch.”

Converting If-Statements to Phi Functions The general process for converting an if-statement to SSA form with phi functions follows these steps:

1. **Version each assignment:** Create a new variable version for every assignment
2. **Track branches:** Identify which variables are assigned in each branch
3. **Insert phi functions:** At the merge point after the if-statement, insert phi functions for any variable that was assigned in either branch

4. **Select the appropriate version:** The phi function chooses between the branch-specific version and the pre-branch version

Consider a general if-statement pattern:

```
x = initial_value
if condition:
    x = new_value
# merge point - which version of x?
use(x)
```

This transforms to:

```
x_0 = initial_value
if condition:
    x_1 = new_value
x_2 = phi(condition, x_1, x_0)
use(x_2)
```

The phi function captures the fact that at the merge point, we need either x_1 (if the condition was true) or x_0 (if the condition was false).

In Z3, phi functions are directly represented using the `If` expression:

$$x_2 = \text{If}(\text{condition}, x_1, x_0)$$

Branch Compilation Strategy For each if-statement, we:

1. Compile the condition to a Z3 boolean expression
2. Create separate compilation contexts for then/else branches
3. Merge the environments using phi-functions
4. Add constraints from both branches

Example: Simple Conditional Consider this Python code:

```
if age >= 18:
    discount = 0
else:
    discount = 10
result = discount * 2
```

Code 4.3: Python conditional statement

The Z3 transformation creates:

```

Condition:  cond = (age ≥ IntVal(18))
Then branch: discount1 = IntVal(0)
Else branch: discount2 = IntVal(10)
Merge:     discount3 = If(cond, discount1, discount2)
Continue:  result0 = discount3 * IntVal(2)

```

Nested Conditionals For nested conditionals, the process recurses:

```

if x > 0:
    if x > 10:
        y = x * 2
    else:
        y = x + 5
else:
    y = 0

```

Code 4.4: Nested conditional example

This generates a nested If structure in Z3:

```

cond1 = (x > IntVal(0))
cond2 = (x > IntVal(10))
y1 = x * IntVal(2)
y2 = x + IntVal(5)
y3 = If(cond2, y1, y2)
y4 = IntVal(0)
y5 = If(cond1, y3, y4)

```

4.3 Loop Unrolling

Since Z3 cannot directly represent arbitrary loops, we employ **bounded loop unrolling** [20, 34] for for loops with statically determinable bounds.

Supported Loop Types The compiler supports two types of bounded loops:

1. **Range loops:** `for i in range(n)` where `n` is a compile-time constant
2. **Container literals:** `for item in [1, 2, 3]` with explicit element lists

Range Loop Unrolling Consider this questionnaire code block:

```
total = 0
for i in range(5):
    total = total + i
    if i == 3:
        bonus = 10
    else:
        bonus = 0
    total = total + bonus
```

Code 4.5: Python for loop with range

The unrolling process creates:

```
total0 = IntVal(0)
Iteration 0:
    i = IntVal(0),    total1 = total0 + IntVal(0)
    bonus0 = If(IntVal(0) = IntVal(3), IntVal(10), IntVal(0))
    total2 = total1 + bonus0
Iteration 1:
    i = IntVal(1),    total3 = total2 + IntVal(1)
    bonus1 = If(IntVal(1) = IntVal(3), IntVal(10), IntVal(0))
    total4 = total3 + bonus1
    ⋮
Iteration 4:
    i = IntVal(4),    total9 = total8 + IntVal(4)
    bonus4 = If(IntVal(4) = IntVal(3), IntVal(10), IntVal(0))
    total10 = total9 + bonus4
```

Container Literal Unrolling For container literals:

```
sum_val = 0
for item in [2, 4, 6, 8]:
    sum_val = sum_val + item
```

Code 4.6: Loop over container literal

This unrolls to:

```
sum_val0 = IntVal(0)
item = IntVal(2), sum_val1 = sum_val0 + IntVal(2)
item = IntVal(4), sum_val2 = sum_val1 + IntVal(4)
item = IntVal(6), sum_val3 = sum_val2 + IntVal(6)
item = IntVal(8), sum_val4 = sum_val3 + IntVal(8)
```

Loop Bounds Safety and Pragmatic Compilation Following our **pragmatic compiler** philosophy, the system prioritizes questionnaire logic analysis over strict compilation failures. When encountering loops that exceed safe bounds:

- Range loops: unroll up to 20 iterations, then stop
- Container literals: process up to 20 elements
- Exceeding bounds triggers a warning, not an error
- Partial results are used for validation and analysis

This approach ensures that questionnaires with large loops can still be analyzed for logical correctness within the unrolled portion, maintaining our primary goal of validating questionnaire logic rather than achieving complete program transformation.

4.4 Integration with Questionnaire Logic

These compilation techniques enable sophisticated questionnaire logic while maintaining Z3 compatibility. Consider a medical risk assessment questionnaire:

Questionnaire Items

- **age**: Integer question “What is your age?” (range: 18-120)
- **diabetes**: Boolean question “Have you been diagnosed with diabetes?” (0=No, 1=Yes)

- **hypertension:** Boolean question “Do you have hypertension?” (0=No, 1=Yes)
- **smoking:** Boolean question “Are you currently a smoker?” (0=No, 1=Yes)
- **assessment:** Computed risk level (1=Low, 2=Medium, 3=High)

Precondition The assessment question only appears for adults:

```
age.outcome >= 18
```

Code Block The code block for risk calculation is as follows:

```
# Calculate risk score based on multiple factors
risk_score = 0

# Age factor
if age.outcome >= 65:
    risk_score = risk_score + 2
elif age.outcome >= 45:
    risk_score = risk_score + 1

# Medical history
for condition in [diabetes.outcome, hypertension.outcome, smoking.outcome]:
    if condition == 1: # Yes answer
        risk_score = risk_score + 1

# Set final assessment
if risk_score >= 4:
    assessment.outcome = 3 # High risk
elif risk_score >= 2:
    assessment.outcome = 2 # Medium risk
else:
    assessment.outcome = 1 # Low risk
```

Code 4.7: Medical risk assessment logic

Postcondition Ensure valid risk assessment range:

```
assessment.outcome >= 1 and assessment.outcome <= 3
```

Z3 Compiled Version The complete transformation to Z3 constraints:

```
// Declare variables
age = Int('age')
diabetes = Int('diabetes')
hypertension = Int('hypertension')
smoking = Int('smoking')
assessment = Int('assessment')

// SSA transformation and risk calculation
risk_score0 = IntVal(0)

// Age factor (nested if-elif-else)
cond_age65 = (age ≥ IntVal(65))
cond_age45 = (age ≥ IntVal(45))
risk_score1 = If(cond_age65,
    risk_score0 + IntVal(2),
    If(cond_age45, risk_score0 + IntVal(1), risk_score0))

// Unrolled loop for medical conditions
risk_score2 = If(diabetes = IntVal(1),
    risk_score1 + IntVal(1), risk_score1)
risk_score3 = If(hypertension = IntVal(1),
    risk_score2 + IntVal(1), risk_score2)
risk_score4 = If(smoking = IntVal(1),
    risk_score3 + IntVal(1), risk_score3)

// Final assessment assignment
cond_high = (risk_score4 ≥ IntVal(4))
cond_medium = (risk_score4 ≥ IntVal(2))
assessment = If(cond_high, IntVal(3),
    If(cond_medium, IntVal(2), IntVal(1)))

// Add constraints
solver.add(age ≥ IntVal(18)) // Precondition
solver.add(age ≤ IntVal(120)) // Range constraint
solver.add(assessment ≥ IntVal(1)) // Postcondition
solver.add(assessment ≤ IntVal(3)) // Postcondition
```

This Z3 representation enables static analysis of:

- **Path coverage:** Verify all risk levels (1, 2, 3) are reachable
- **Consistency:** Ensure no contradictory conditions exist
- **Boundary analysis:** Validate edge cases (e.g., age=45, age=65, risk_score=2, risk_score=4)
- **Postcondition satisfaction:** Prove assessment always falls within valid range

4.5 Z3 Compilation for Complex Types

The extended type system (Section 3.7) introduces significant challenges for constraint compilation. While scalar variables map directly to Z3 integer variables, vectors and matrices require more sophisticated representations using Z3’s array theory and quantified formulas. This section describes how complex questionnaire types are encoded into Z3 constraints while maintaining the solver’s ability to efficiently check satisfiability.

The compilation process must handle three key challenges:

- **Data structure representation:** Encoding vectors and matrices as Z3 arrays with appropriate index domains
- **Constraint translation:** Converting high-level constraints (e.g., “all different”, “sum equals 100”) into quantified formulas
- **Computational tractability:** Balancing expressiveness with solver performance, particularly for constraints involving nested quantifiers

We present the compilation strategies for each complex type, demonstrating how Python expressions operating on vectors and matrices are systematically transformed into equivalent Z3 constraints that preserve the intended semantics while remaining decidable.

Vector Representation in Z3 Vectors are represented using Z3’s array theory, where arrays map integer indices to integer values:

```
1 # Declare a vector item with 5 components
2 vector_item = Array('S_i', IntSort(), IntSort())
3
4 # Domain constraints: each component in [1, 10]
5 domain_constraints = And([
6     And(1 <= vector_item[j], vector_item[j] <= 10)
7     for j in range(5)
8 ])
9
10 # Example: Ranking constraint (all distinct)
11 ranking_constraint = Distinct([vector_item[j] for j in range(5)])
```

Code 4.8: Vector item representation in Z3

Matrix Representation in Z3 Matrices use nested arrays, where the outer array maps row indices to inner arrays (representing rows):

```
1 # Declare a 3x4 matrix item
2 matrix_item = Array('S_i', IntSort(),
3                     ArraySort(IntSort(), IntSort()))
4
5 # Access element at row j, column k
6 element_j_k = matrix_item[j][k]
7
8 # Domain constraints for all elements
9 domain_constraints = And([
10     And([
11         And(0 <= matrix_item[j][k], matrix_item[j][k] <= 100)
12         for k in range(4)
13     ])
14     for j in range(3)
15 ])
```

Code 4.9: Matrix item representation in Z3

4.5.1 Compiling Complex Type Operations

The Python-to-Z3 compiler extends to handle vector and matrix operations:

Vector Subscript Access Python code accessing vector components translates directly to array access in Z3:

```
# If first choice is option 1, set followup
if ranking.outcome[0] == 1:
    followup.outcome = 1
else:
    followup.outcome = 0
```

Code 4.10: Python code with vector access

The Z3 transformation with SSA versioning:

```
// Declare vector as Z3 array
ranking = Array('ranking', IntSort(), IntSort())
followup = Int('followup')

// SSA transformation
cond = (ranking[0] == IntVal(1))
followup1 = IntVal(1)
followup2 = IntVal(0)
followup3 = If(cond, followup1, followup2)
solver.add(followup == followup3)
```

Aggregate Operations Sum operations over vector components compile to Z3 sum expressions:

```
# Calculate total allocation (must equal 100)
total = 0
for j in range(4):
    total = total + allocation.outcome[j]
```

Code 4.11: Python code with vector sum

The Z3 transformation with loop unrolling and SSA:

```
// Declare allocation vector
allocation = Array('allocation', IntSort(), IntSort())

// Unrolled loop with SSA versioning
total0 = IntVal(0)
total1 = total0 + allocation[0]
total2 = total1 + allocation[1]
total3 = total2 + allocation[2]
total4 = total3 + allocation[3]

// Alternatively, using Z3's Sum function
total = Sum([allocation[j] for j in range(4)])
```

Matrix Operations Matrix operations require nested array access and iteration:

```
# Sum all ratings in first row (first product)
row_sum = 0
for j in range(4):
    row_sum = row_sum + grid.outcome[0][j]
```

Code 4.12: Python code with matrix row sum

The Z3 transformation with nested array access:

```
// Declare matrix as nested Z3 arrays
grid = Array('grid', IntSort(),
             ArraySort(IntSort(), IntSort()))

// Unrolled loop for row sum with SSA
row_sum0 = IntVal(0)
row_sum1 = row_sum0 + grid[0][0]
row_sum2 = row_sum1 + grid[0][1]
row_sum3 = row_sum2 + grid[0][2]
row_sum4 = row_sum3 + grid[0][3]

// Alternative using Z3's Sum
row_sum = Sum([grid[0][j] for j in range(4)])
```

4.5.2 Implementation Notes

Complex types are built into the QML (Questionnaire Markup Language) type system. Each item's `outcome` attribute can be:

- **Integer**: For scalar questions (single value)
- **List[Integer]**: For vector questions (rankings, multiple selections)
- **List[List[Integer]]**: For matrix questions (grids, comparison tables)

Because Python uses dynamic typing, the `outcome` attribute can be any of these types at runtime. The questionnaire framework determines the actual type based on the item's configuration in the QML specification.

The Python-to-Z3 compiler recognizes complex type patterns. Some examples are:

- `item.outcome[i]` → Vector access
- `item.outcome[i][j]` → Matrix access
- `sum(item.outcome)` → Vector sum
- `len(item.outcome)` → Vector/matrix dimension

Constraint Complexity Growth Complex types increase constraint complexity:

- **Vector of size k** : Multiplies constraints by factor of k
- **Matrix of size $m \times n$** : Multiplies constraints by factor of $m \cdot n$
- **Distinct constraints**: Add $\binom{k}{2}$ inequality constraints for k elements

To manage increased complexity:

1. **Lazy constraint generation**: Only instantiate constraints for accessed components
2. **Symmetry breaking**: For symmetric structures, reduce redundant constraints

3. **Domain pruning:** Use domain knowledge to eliminate impossible combinations early
4. **Incremental solving:** Check satisfiability incrementally as constraints are added

Complete QML examples demonstrating vector and matrix item types can be found in Appendix A.

4.6 Compiler Characteristics

AST Processing Pipeline The Python-to-Z3 compiler follows a structured pipeline based on established compiler design principles [35]:

1. **Parsing:** Convert Python code to Abstract Syntax Tree (AST)
2. **Validation:** Ensure only supported language constructs are used
3. **SSA Transformation:** Convert to single static assignment form
4. **Constraint Generation:** Translate AST nodes to Z3 constraints
5. **Optimization:** Simplify generated constraints where possible

Supported Python Subset The compiler supports a carefully chosen subset of Python that covers common questionnaire logic patterns:

- **Arithmetic Operations:** `+`, `-`, `*`, `//`, `%`
- **Comparison Operations:** `==`, `!=`, `<`, `<=`, `>`, `>=`
- **Boolean Operations:** `and`, `or`, `not`
- **Conditional Statements:** `if/elif/else`
- **Bounded Loops:** `for` loops with constant bounds
- **Variable Assignment:** with SSA transformation
- **List Literals:** `[1, 2, 3]` for loop iteration
- **Attribute Access:** `question.outcome` for questionnaire data

Future Extensions Potential improvements include:

- **Function Inlining:** Support for simple user-defined functions
- **Array Theory:** Better support for list operations using Z3's array theory
- **String Constraints:** Integration with Z3's string theory for text processing
- **Modular Analysis:** Breaking large questionnaires into analyzable components
- **Exception Handling:** Support for try/except constructs
- **Complex Data Structures:** Not just lists and matrices, but also dictionaries, sets, and other data structures.

Chapter 5

Comprehensive Questionnaire Validation

This chapter clarifies the relationship between a global satisfiability check of the questionnaire and the per-item implication checks used in the implementation. We formalize both notions and prove what is (and is not) equivalent, with practical consequences for validation.

5.1 Setup

Recall from Definition 3 that each outcome variable S_i has an associated domain constraint $D_i(S_i)$. Let $B := \bigwedge_{i=1}^n D_i(S_i)$ denote the conjunction of all domain constraints in the questionnaire. For each item I_i , let P_i be its precondition and Q_i its postcondition, expressed as Boolean formulas over the answers $\mathbf{S} = (S_1, \dots, S_n)$.

Definition 9 (Global Satisfiability Formula). The **global satisfiability formula** for a questionnaire is:

$$\mathcal{F} := B \wedge \bigwedge_{i=1}^n (P_i \Rightarrow Q_i)$$

We say the questionnaire passes the global satisfiability check if $\text{SAT}(\mathcal{F})$ holds.

Definition 10 (Witness Formula). For an item I_i , the **witness formula** is:

$$W_i := B \wedge P_i \wedge \neg Q_i$$

If $\text{SAT}(W_i)$, then there exists an assignment satisfying the domain constraints where item I_i is reached (precondition P_i holds) but the postcondition Q_i is violated. Such an assignment demonstrates that the item can fail validation.

The key diagnostic question is whether W_i is satisfiable: $\text{UNSAT}(W_i)$ means item I_i is *locally valid* (whenever it is reached, its postcondition must hold), while $\text{SAT}(W_i)$ indicates a potential violation.

5.2 Key Relations

The following theorems capture the exact relationship between the global check and the per-item checks.

Theorem 1 (Soundness of Per-Item Validity). *If $\text{SAT}(B)$ and for every $i \in \{1, \dots, n\}$ we have $\text{UNSAT}(W_i)$, then $\text{SAT}(\mathcal{F})$.*

Proof. Assume $\text{SAT}(B)$ and $\text{UNSAT}(W_i)$ for all $i \in \{1, \dots, n\}$, where $W_i = B \wedge P_i \wedge \neg Q_i$.

Step 1: Transform unsatisfiability to entailment for each item.

For each i , we have $\text{UNSAT}(B \wedge P_i \wedge \neg Q_i)$.

By definition of unsatisfiability:

$$\nexists \mathbf{a} : \mathbf{a} \models (B \wedge P_i \wedge \neg Q_i)$$

Equivalently, by the quantifier negation rule (i.e., $\neg \exists x P(x) \equiv \forall x \neg P(x)$):

$$\forall \mathbf{a} : \mathbf{a} \not\models (B \wedge P_i \wedge \neg Q_i)$$

By the definition of satisfaction for conjunction (an assignment fails to satisfy a conjunction if it fails to satisfy at least one conjunct), this means for all assignments $\mathbf{a}$:

$$\mathbf{a} \not\models B \text{ or } \mathbf{a} \not\models P_i \text{ or } \mathbf{a} \not\models \neg Q_i$$

Note that $\mathbf{a} \not\models \neg Q_i$ is equivalent to $\mathbf{a} \models Q_i$ (by double negation).

Rewriting the disjunction using the logical equivalence $(\neg A \vee \neg B \vee C) \equiv ((A \wedge B) \Rightarrow C)$, we get: for all assignments $\mathbf{a}$:

$$\text{if } \mathbf{a} \models B \wedge P_i, \text{ then } \mathbf{a} \models Q_i$$

by definition of $\models$, this is equivalent to:

$$B \wedge P_i \models Q_i$$

By the semantic deduction theorem (moving P_i to the right), this is the definition of:

$$B \models (P_i \Rightarrow Q_i)$$

Step 2: Combine entailments for all items.

We have established that $B \models (P_i \Rightarrow Q_i)$ for each $i \in \{1, \dots, n\}$.

By the monotonicity of entailment: if $B \models \phi_i$ for each i , then $B \models \bigwedge_i \phi_i$.

Therefore:

$$B \models \bigwedge_{i=1}^n (P_i \Rightarrow Q_i)$$

Step 3: Construct a model for $\mathcal{F}$.

By hypothesis, $\text{SAT}(B)$, so there exists an assignment $\mathbf{a}$ such that $\mathbf{a} \models B$.

From Step 2, we have $B \models \bigwedge_{i=1}^n (P_i \Rightarrow Q_i)$.

By the definition of semantic entailment, every model of B is also a model of $\bigwedge_{i=1}^n (P_i \Rightarrow Q_i)$.

Since $\mathbf{a} \models B$, we have $\mathbf{a} \models \bigwedge_{i=1}^n (P_i \Rightarrow Q_i)$.

Therefore:

$$\mathbf{a} \models B \wedge \bigwedge_{i=1}^n (P_i \Rightarrow Q_i) = \mathcal{F}$$

Hence, $\text{SAT}(\mathcal{F})$. □

Theorem 2 (Local Invalidity Does Not Imply Global Unsatisfiability). *If $\text{SAT}(W_i)$ for some i , it does not follow that $\text{UNSAT}(\mathcal{F})$.*

The witness W_i only shows that there *exists* a violating model of B ; the global check asks whether there *exists* a (possibly different) model that satisfies all implications simultaneously.

Example. Consider a two-item questionnaire:

- I_1 : “What is your age?” with $S_1 \in [0, 120]$, $P_1 = \text{true}$, $Q_1 = \text{true}$
- I_2 : “Years of driving experience?” with $S_2 \in [0, 100]$, $P_2 = (S_1 \geq 16)$, $Q_2 = (S_2 \leq S_1 - 16)$

The postcondition Q_2 ensures driving experience cannot exceed years since turning 16. Consider the witness formula for I_2 :

$$W_2 = B \wedge P_2 \wedge \neg Q_2 = B \wedge (S_1 \geq 16) \wedge (S_2 > S_1 - 16)$$

This is satisfiable: $S_1 = 20$, $S_2 = 10$ satisfies W_2 (claiming 10 years of experience at age 20, which violates Q_2 since $10 > 20 - 16 = 4$). So $\text{SAT}(W_2)$ —the per-item check reports item I_2 as *locally invalid*.

However, the global formula $\mathcal{F}$ is still satisfiable. Consider $S_1 = 30$, $S_2 = 5$:

- $P_2 = (30 \geq 16) = \text{true}$, and $Q_2 = (5 \leq 30 - 16) = (5 \leq 14) = \text{true}$
- Thus $(P_2 \Rightarrow Q_2) = \text{true}$

This assignment satisfies $\mathcal{F}$, so $\text{SAT}(\mathcal{F})$ despite $\text{SAT}(W_2)$. The local invalidity witness shows that *some* respondents could violate the postcondition, but the questionnaire still has valid completions.

Theorem 3 (Accumulated Constraints Can Cause Global Unsatisfiability). *There exist questionnaires where no item is INFEASIBLE (each $\text{SAT}(B \wedge P_i \wedge Q_i)$), yet the global formula $\mathcal{F}$ is unsatisfiable.*

Per-item checks examine each postcondition independently against the domain constraints B . However, during questionnaire execution, items are evaluated in dependency order, and postconditions from earlier items must be satisfied before proceeding to later items. These accumulated constraints are not visible to per-item checks.

Example. Consider a two-item questionnaire where I_2 depends on S_1 :

- I_1 : “Rate your experience (1–100)” with $S_1 \in [1, 100]$, $P_1 = \text{true}$, $Q_1 = (S_1 > 50)$
- I_2 : “Confirm your rating” with $S_2 \in \{0, 1\}$, $P_2 = \text{true}$, $Q_2 = (S_1 < 30)$

Since Q_2 references S_1 , the dependency graph requires $I_1 \rightarrow I_2$ ordering. The respondent must satisfy Q_1 (i.e., $S_1 > 50$) before proceeding to I_2 .

Per-item analysis:

- $W_1 = B \wedge \text{true} \wedge (S_1 \leq 50)$ is SAT (e.g., $S_1 = 25$) $\Rightarrow$ CONSTRAINING

- $W_2 = B \wedge \text{true} \wedge (S_1 \geq 30)$ is SAT (e.g., $S_1 = 50$) $\Rightarrow$ CONSTRAINING
- Neither item is INFEASIBLE: $\text{SAT}(B \wedge P_1 \wedge Q_1)$ and $\text{SAT}(B \wedge P_2 \wedge Q_2)$

Global analysis:

$$\mathcal{F} = B \wedge (\text{true} \Rightarrow S_1 > 50) \wedge (\text{true} \Rightarrow S_1 < 30) = B \wedge (S_1 > 50) \wedge (S_1 < 30)$$

This is unsatisfiable—no value of S_1 can simultaneously satisfy $S_1 > 50$ and $S_1 < 30$. Therefore $\text{UNSAT}(\mathcal{F})$.

The per-item checks miss this conflict because they evaluate each postcondition against all possible values of S_1 , not accounting for the fact that Q_1 must already be satisfied when I_2 is reached.

5.3 Path-Based Validation

Path-based validation addresses a limitation of the global formula $\mathcal{F}$: the use of implications ($P_i \Rightarrow Q_i$) that are vacuously true when preconditions are false. This section formalizes the path-based approach and proves its relationship to global validation.

Definition 11 (Execution Path). An **execution path** π through a questionnaire is a sequence of items $(I_{k_1}, I_{k_2}, \dots, I_{k_m})$ respecting the dependency order, where each item's precondition is satisfied given the accumulated constraints from preceding items.

Definition 12 (Path Constraint Formula). For an execution path $\pi = (I_{k_1}, \dots, I_{k_m})$, the **path constraint formula** is:

$$\Phi_\pi := B \wedge \bigwedge_{j=1}^m (P_{k_j} \wedge Q_{k_j})$$

The path is **feasible** if $\text{SAT}(\Phi_\pi)$.

Note the key difference from the global formula: path constraints use conjunctions ($P_i \wedge Q_i$) for items on the path, not implications ($P_i \Rightarrow Q_i$). This ensures that preconditions are actually satisfiable, not just vacuously handled.

Theorem 4 (Global Validity is Necessary for Path Validity). *If $\text{UNSAT}(\mathcal{F})$, then for every execution path π , we have $\text{UNSAT}(\Phi_\pi)$.*

Proof. For any path $\pi = (I_{k_1}, \dots, I_{k_m})$, if $\text{SAT}(\Phi_\pi)$, then there exists an assignment $\mathbf{a}$ satisfying $B \wedge \bigwedge_{j=1}^m (P_{k_j} \wedge Q_{k_j})$.

For items on the path: $\mathbf{a} \models P_{k_j}$ and $\mathbf{a} \models Q_{k_j}$, so $\mathbf{a} \models (P_{k_j} \Rightarrow Q_{k_j})$.

For items not on the path: their preconditions are false under $\mathbf{a}$ (otherwise they would be on the path), so $(P_i \Rightarrow Q_i)$ is vacuously true.

Therefore $\mathbf{a} \models \mathcal{F}$, contradicting $\text{UNSAT}(\mathcal{F})$. $\square$

Theorem 5 (Global Validity Does Not Guarantee All Items Reachable). *There exist questionnaires where $\text{SAT}(\mathcal{F})$ but some item I_i with $P_i \neq \text{false}$ is unreachable on every feasible path.*

Example. Consider:

- $I_1: S_1 \in [1, 100], P_1 = \text{true}, Q_1 = (S_1 > 80)$
- $I_2: S_2 \in \{0, 1\}, P_2 = (S_1 < 50), Q_2 = \text{true}$

The dependency graph requires $I_1 \rightarrow I_2$ (since P_2 references S_1).

Per-item analysis:

- $P_2 = (S_1 < 50)$ is CONDITIONAL: $\text{SAT}(B \wedge S_1 < 50)$ with $S_1 = 25$
- $Q_1 = (S_1 > 80)$ is CONSTRAINING: $\text{SAT}(W_1)$ with $S_1 = 25$

Global analysis:

$$\mathcal{F} = B \wedge (\text{true} \Rightarrow S_1 > 80) \wedge ((S_1 < 50) \Rightarrow \text{true}) \equiv B \wedge (S_1 > 80)$$

This is satisfiable (e.g., $S_1 = 90$), so $\text{SAT}(\mathcal{F})$.

Path-based analysis: Any path including I_2 requires:

$$\Phi_\pi = B \wedge (P_1 \wedge Q_1) \wedge (P_2 \wedge Q_2) \equiv B \wedge (S_1 > 80) \wedge (S_1 < 50)$$

This is unsatisfiable—no value of S_1 can satisfy both $S_1 > 80$ and $S_1 < 50$.

Therefore, I_2 is classified as CONDITIONAL by per-item analysis (reachable when $S_1 < 50$), the global check passes, but I_2 is actually unreachable on any valid execution path. This is **dead code** that path-based validation detects.

Practical Path Enumeration. In practice, path enumeration leverages the dependency graph structure constructed in Section 3.6. The topological ordering com-

puted by Kahn’s algorithm (Algorithm 3) determines the evaluation order: items with no dependencies can be traversed in any order, while dependent items must follow their predecessors. The path-based check:

1. Uses the dependency graph from Section 3.6 to identify predecessor relationships
2. For each item I_i with a **CONDITIONAL** precondition, constructs the accumulated constraint formula from all predecessor items in the topological order
3. Checks if the item remains reachable under accumulated constraints

Definition 13 (Accumulated Reachability). For item I_i , let $\text{Pred}(i) = \{j : I_j \prec I_i\}$ be the set of predecessor items, where $I_j \prec I_i$ denotes that I_j precedes I_i in the topological order. The **accumulated constraint formula** is:

$$\mathcal{A}_i := B \wedge \bigwedge_{j \in \text{Pred}(i)} (P_j \Rightarrow Q_j)$$

Item I_i is **accumulated-reachable** if $\text{SAT}(\mathcal{A}_i \wedge P_i)$.

Example. Consider a three-item questionnaire:

- I_1 : “Annual income” with $S_1 \in [0, 1000000]$, $P_1 = \text{true}$, $Q_1 = (S_1 \geq 50000)$
- I_2 : “Tax bracket” with $S_2 \in \{1, 2, 3\}$, $P_2 = \text{true}$, $Q_2 = \text{true}$
- I_3 : “Low-income assistance” with $S_3 \in \{0, 1\}$, $P_3 = (S_1 < 30000)$, $Q_3 = \text{true}$

The dependency graph has $I_1 \rightarrow I_3$ (since P_3 references S_1), while I_2 is independent.

Per-item analysis:

- $P_3 = (S_1 < 30000)$ is **CONDITIONAL**: $\text{SAT}(B \wedge S_1 < 30000)$ holds

Accumulated reachability for I_3 :

The predecessor set is $\text{Pred}(3) = \{1\}$ (only I_1 precedes I_3 in the dependency order; I_2 is independent and not a predecessor).

$$\mathcal{A}_3 = B \wedge (P_1 \Rightarrow Q_1) = B \wedge (\text{true} \Rightarrow S_1 \geq 50000) = B \wedge (S_1 \geq 50000)$$

Now check accumulated reachability:

$$\mathcal{A}_3 \wedge P_3 = B \wedge (S_1 \geq 50000) \wedge (S_1 < 30000)$$

This is unsatisfiable—no income value can be both ≥ 50000 and < 30000 . Therefore I_3 is **not accumulated-reachable**.

The questionnaire designer intended I_3 (low-income assistance) to appear for respondents with income below \$30,000, but Q_1 requires income $\geq \$50,000$ to proceed. This is dead code that accumulated reachability analysis detects.

If $\text{UNSAT}(\mathcal{A}_i \wedge P_i)$ for a CONDITIONAL item, the item is dead code—its precondition can never be satisfied given the constraints imposed by predecessor items.

5.4 Validation Hierarchy

Questionnaire validation can be performed at three levels of increasing thoroughness and computational cost:

1. **Per-item validation** (Chapter 3): For each item I_i , check independently:
 - *Reachability*: Classify P_i as ALWAYS, NEVER, or CONDITIONAL using $\text{SAT}(B \wedge P_i)$
 - *Validity*: Classify Q_i as TAUTOLOGICAL, CONSTRAINING, or INFEASIBLE using the witness formula W_i

This is the fastest level—each item can be checked independently and in parallel.

2. **Global validation**: Check $\text{SAT}(\mathcal{F})$ where $\mathcal{F} = B \wedge \bigwedge_i (P_i \Rightarrow Q_i)$. This verifies that at least one valid questionnaire completion exists, but uses implications that are vacuously true when preconditions are false.
3. **Path-based validation**: Enumerate all execution paths through the questionnaire and check each path's accumulated constraints. This is the most

thorough level, detecting items that become unreachable due to accumulated postconditions (dead code).

Relationships Between Levels. The three validation levels form a hierarchy with precise logical relationships:

- **Per-item passes $\Rightarrow$ Global passes:** By Theorem 1, if all W_i are UNSAT (all postconditions TAUTOLOGICAL), then $\text{SAT}(\mathcal{F})$ is guaranteed. No further checks needed.
- **Global fails $\Rightarrow$ Path-based fails:** If $\text{UNSAT}(\mathcal{F})$, then no execution path can be valid. The global formula is a necessary condition for questionnaire validity.
- **Global passes $\nRightarrow$ All paths valid:** The global check can pass while some items are unreachable, because implications $(P_i \Rightarrow Q_i)$ are vacuously true when P_i is false.

When Each Level Suffices.

- **Per-item validation suffices** when all postconditions are TAUTOLOGICAL. The questionnaire has no constraining logic, so any answer combination is valid.
- **Global validation suffices** when you only need to verify that *some* valid completion exists. Dead code (unreachable items) is acceptable.
- **Path-based validation is needed** when you require that *every* conditional item is actually reachable on at least one valid path. This detects design errors where accumulated postconditions make items unreachable.

Summary of Validation Results.

- **All W_i UNSAT:** All postconditions are TAUTOLOGICAL. By Theorem 1, global and path-based validity are guaranteed—no further checks needed.
- **For some i , $\text{SAT}(W_i)$, $\text{UNSAT}(\mathcal{F})$:** Accumulated CONSTRAINING postconditions conflict (Theorem 3). By Theorem 4, no execution path is valid. This is a design error.

- **For some i , $\text{SAT}(W_i)$, $\text{SAT}(\mathcal{F})$, all items accumulated-reachable:** The questionnaire is fully valid—completable with no dead code.
- **For some i , $\text{SAT}(W_i)$, $\text{SAT}(\mathcal{F})$, some items not accumulated-reachable:** The questionnaire is completable (Theorem 2), but contains dead code (Theorem 5). Report as warning.
- **Some item INFEASIBLE:** The postcondition cannot be satisfied when the item is reached—design error requiring fix.
- **Some item NEVER reachable (per-item):** Precondition is unsatisfiable even without accumulated constraints—design error.

Chapter 6

Conclusion

This thesis has presented a formal approach to validating questionnaire logic through Satisfiability Modulo Theories (SMT) solvers. By transforming questionnaire constraints into mathematical formulas and leveraging automated theorem proving, we can provide rigorous guarantees about questionnaire behavior that would be impossible to achieve through manual validation or traditional testing approaches.

6.1 Summary of Contributions

Our work makes several key contributions to the field of questionnaire validation and formal methods:

Framework for Questionnaire Analysis We developed a comprehensive mathematical framework for representing questionnaire logic that includes:

- **Formal Definitions:** Precise mathematical definitions of questionnaires, items, preconditions, postconditions, and answer dependencies
- **Questionnaire Markup Language (QML):** A domain-specific language for expressing questionnaires in a single text file with declarative constraints and imperative code blocks to enable AI-aided questionnaire generation
- **Item Reachability Analysis:** Classification of items through precondition analysis as always reachable (precondition is always satisfiable), conditionally reachable (precondition depends on prior answers), or never reachable (precondition is unsatisfiable)

- **Item Validity Analysis:** Classification of items through postcondition analysis as tautological (postcondition always holds when item is reached), constraining (postcondition meaningfully restricts answer values), or infeasible (postcondition cannot be satisfied)
- **Dependency Analysis:** Graph-based methods for detecting circular dependencies and analyzing question reachability
- **Consistency Checking:** Formal criteria for determining whether questionnaire logic is internally consistent

This framework provides a solid theoretical foundation for questionnaire validation that can be applied beyond our specific SMT-based implementation. Notably, our analysis of the relationship between per-item validity checks and global satisfiability (Chapter 5) clarifies when local validation suffices and when global analysis is necessary, providing practical guidance for validation strategies.

6.2 Impact and Significance

Improving Questionnaire Quality Our approach directly addresses the quality challenges introduced by automated questionnaire generation:

- **Early Detection:** Logical problems are identified during questionnaire design rather than after deployment
- **Complete Coverage:** Formal analysis ensures that all questions can be reached and all logic paths are valid
- **Objective Validation:** Mathematical guarantees replace subjective manual review processes
- **Cost Reduction:** Automated validation reduces the time and effort required for questionnaire testing

Enabling AI-Aided Generation Current AI tools struggle with complex survey generation because Large Language Models (LLMs) face an overwhelming cognitive burden when attempting to simultaneously:

- Generate semantically meaningful questions
- Track conditional dependencies between questions
- Ensure logical consistency across all paths
- Maintain correct question ordering and flow
- Avoid contradictions and unreachable dead ends

This cognitive overload effectively limits AI-generated surveys to simple, linear questionnaires of 10–20 questions—far below the complexity required for enterprise applications, regulatory compliance, or sophisticated research instruments.

Our system fundamentally transforms this landscape through **separation of concerns**:

- **LLMs Focus on Content Generation:** AI systems concentrate solely on creating relevant, domain-appropriate questions based on requirements, without worrying about logical structure or dependencies
- **Mathematical Validation Ensures Coherence:** The Z3 SMT solver automatically handles all logical complexity—ensuring consistency, detecting contradictions, identifying unreachable paths, and optimizing question flow through formal verification
- **Declarative Specification Enables Complexity:** Questions and their conditions are specified declaratively, allowing to reorder items without breaking the logic

This architectural separation unlocks a new capability: LLMs can now generate enterprise-grade questionnaires with hundreds of interconnected questions, complex branching logic, and sophisticated conditional dependencies—achieving a level of complexity that is simply impossible with current AI survey tools. The AI doesn’t need to “think” about question ordering or logical conditions; it merely needs to generate relevant content while our formal validation system ensures mathematical correctness.

6.3 Limitations and Challenges

Technical Limitations Our current approach has several technical limitations that constrain its applicability:

- **Python Subset:** The compiler supports only a restricted subset of Python, limiting the complexity of embedded code blocks
- **Bounded Analysis:** Loop unrolling is limited to small, statically determinable bounds
- **Scalability:** Very large questionnaires may challenge SMT solver performance
- **Dynamic Behavior:** Runtime-dependent questionnaire behavior cannot be fully analyzed statically

Theoretical Limitations Some fundamental limitations stem from the underlying theory:

- **Decidability:** Some questionnaire logic patterns may lead to undecidable SMT problems
- **Completeness:** Our analysis may miss subtle bugs that require deeper semantic understanding
- **Abstraction:** The translation from Python to Z3 necessarily involves some semantic loss
- **Model Limitations:** Real questionnaire behavior may deviate from our formal models

6.4 Future Research Directions

Technical Enhancements Several technical improvements could extend the capabilities of our approach:

- **Extended Language Support:** Expanding the Python subset to support more complex programming constructs such as functions, classes, and exception handling

- **Dynamic Analysis Integration:** Combining static SMT-based analysis with dynamic testing to catch runtime-dependent issues
- **Incremental Validation:** Developing techniques for efficiently revalidating questionnaires after small changes
- **Optimization Techniques:** Improving SMT solver performance through domain-specific optimizations and constraint simplification

Application Domains The techniques developed in this thesis could be applied to critical decision-support systems across various domains:

- **Automated Compliance Audits:** Using formal questionnaire logic to ensure regulatory compliance by systematically checking that all required conditions are met, with mathematical guarantees that no compliance path is overlooked and all regulatory constraints are satisfied.
- **Legal Case Discovery:** Implementing dynamic question generation for legal interrogation support, where each answer influences subsequent questions to efficiently navigate complex legal decision trees, helping legal professionals identify relevant case factors and precedents through minimal questioning.
- **Emergency Medical Triage:** Developing diagnostic helper tools for first responders and emergency call operators that use adaptive questioning to quickly identify critical medical conditions with the minimum number of questions, ensuring that life-threatening conditions are identified rapidly while maintaining diagnostic accuracy.
- **Critical System Verification and Debugging:** Ensuring logically sound operational procedures for safety-critical systems where checklist errors can have catastrophic consequences. This includes pre-launch verification for space shuttles, startup sequences for particle accelerators, safety protocols for nuclear reactor experiments, and diagnostic procedures for distributed computing infrastructures. By modeling these critical checklists as formal questionnaires with conditional logic, we can mathematically verify that no critical step can be bypassed, all safety interlocks are properly enforced, and every possible system state leads to appropriate verification steps. This transforms traditional

paper checklists and operational procedures into formally verified protocols with mathematical guarantees that prevent human error in life-critical scenarios.

6.5 Final Thoughts

The proliferation of AI-generated questionnaires and complex conditional logic in surveys presents both opportunities and challenges. While AI can rapidly generate sophisticated questionnaire structures, ensuring their logical correctness remains a critical challenge. This thesis addresses this gap by demonstrating how formal methods—specifically SMT-based validation—can provide mathematical guarantees about questionnaire behavior.

Looking forward, as questionnaires evolve from simple data collection instruments to sophisticated decision-support systems—whether for medical triage, legal discovery, or regulatory compliance—the need for formal validation will only intensify. The techniques presented in this thesis provide a foundation for ensuring these critical systems behave correctly, completely, and consistently.

This thesis demonstrates that formal methods need not be confined to safety-critical systems or theoretical computer science. By applying these techniques to the practical problem of questionnaire validation, we show that formal verification can be made accessible, useful, and even indispensable for ensuring the quality of AI-generated and human-designed logical structures alike.

Bibliography

- [1] Joshua Lerner. *The Promise and Pitfalls of AI-Augmented Survey Research*. Tech. rep. Discusses challenges of AI-generated questionnaires including logical consistency and validation issues, Accessed: 2025-11-26. NORC at the University of Chicago, Oct. 2024. URL: <https://www.norc.org/research/library/promise-pitfalls-ai-augmented-survey-research.html>.
- [2] SurveyMonkey Research Team. “Evaluating the Quality of Questionnaires Created with SurveyMonkey’s Build with AI”. In: *Proceedings of the 79th Annual Conference of the American Association for Public Opinion Research (AAPOR)*. Empirical evaluation of AI-generated survey quality and logical flow issues. 2024.
- [3] “The Questionable Utility of LLMs for Open-Ended Question Design Research”. In: *Proceedings of the 79th Annual Conference of the American Association for Public Opinion Research (AAPOR)*. Critical analysis of limitations in LLM-generated survey questions, including lack of logical flow. 2024.
- [4] Francisco Olivos and Minhui Liu. “ChatGPTTest: Opportunities and Cautionary Tales of Utilizing AI for Questionnaire Pretesting”. In: *Field Methods* (2024). Explores AI capabilities and limitations in questionnaire validation and pretesting, arXiv:2405.06329. DOI: 10.1177/1525822X241280574.
- [5] Leon Willenborg. “Testing the logical structure of questionnaires”. In: *Proceedings of the ASA Section on Survey Research Methods*. Demonstrates that checking questionnaire logical consistency is NP-complete. American Statistical Association. 1995, pp. 1032–1037.
- [6] Claude Picard. *Théorie des questionnaires*. Foundational work on formal questionnaire theory. Paris: Gauthier-Villars, 1965.

- [7] Ivan P Fellegi and David Holt. “A systematic approach to automatic edit and imputation”. In: *Journal of the American Statistical Association* 71.353 (1976), pp. 17–35. DOI: 10.1080/01621459.1976.10481472.
- [8] Leon Willenborg. “Computational Aspects of Survey Data Processing”. Foundational work on formal logical structure of questionnaires, Accessed: 2025-11-26. PhD thesis. Voorburg, Netherlands: Netherlands Central Bureau of Statistics, 1988. URL: <https://download.cbs.nl/proefschriften/dissertation%20willenborg.pdf>.
- [9] Jeffrey T Fagan and Brian V Greenberg. “Using graph theory to analyze skip patterns in questionnaires”. In: *Proceedings of the Section on Survey Research Methods*. American Statistical Association. 1988, pp. 632–637.
- [10] Jelke G Bethlehem and Anco Hundepool. “TADEQ: A tool for documentation and analysis of electronic questionnaires”. In: *Journal of Official Statistics* 20.2 (2004), pp. 233–263.
- [11] David Elliott. “The application of graph theory to the development and testing of survey instruments”. In: *Survey Methodology* 38.2 (2012), pp. 195–206.
- [12] Ioana Schiopu-Kratina et al. “Survey questionnaires and graphs”. In: *Electronic Journal of Statistics* 9.2 (2015), pp. 2005–2035. DOI: 10.1214/15-EJS1067.
- [13] Charles F Manski and Francesca Molinari. “Skip sequencing: A decision problem in questionnaire design”. In: *The Annals of Applied Statistics* 2.1 (2008), pp. 264–285. DOI: 10.1214/07-A0AS134.
- [14] Griffith Feeney and Samuel Feeney. “On the logical structure of census and survey questionnaires”. In: *Genus* 75.1 (2019). Recent work on computer-readable questionnaire representation and path enumeration, pp. 1–23. DOI: 10.1186/s41118-019-0065-y.
- [15] Isaac Nassi and Ben Shneiderman. “Flowchart Techniques for Structured Programming”. In: *ACM SIGPLAN Notices* 8.8 (1973). Introduced structured diagrams as an alternative to flowcharts for visualizing program logic, pp. 12–26. DOI: 10.1145/953349.953350.

- [16] Edsger W Dijkstra. “Notes on Structured Programming”. In: *Structured Programming*. Ed. by O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare. Contains the famous observation that “Program testing can be used to show the presence of bugs, but never to show their absence”. London: Academic Press, 1972, pp. 1–82. ISBN: 9780122005503.
- [17] C. A. R. Hoare. “An axiomatic basis for computer programming”. In: *Communications of the ACM* 12.10 (1969), pp. 576–580. DOI: 10.1145/363235.363259.
- [18] Leonardo de Moura and Nikolaj Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. TACAS 2008. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.
- [19] Ron Cytron et al. “Efficiently computing static single assignment form and the control dependence graph”. In: *ACM Transactions on Programming Languages and Systems* 13.4 (1991), pp. 451–490. DOI: 10.1145/115372.115320.
- [20] Edmund Clarke et al. “Bounded model checking using satisfiability solving”. In: *Formal methods in system design* 19.1 (2001), pp. 7–34. DOI: 10.1023/A:1011276507260.
- [21] Don A Dillman, Jolene D Smyth, and Leah Melani Christian. *Internet, phone, mail, and mixed-mode surveys: the tailored design method*. 4th. John Wiley & Sons, 2014. ISBN: 9781118456149.
- [22] Hamed Taherdoost. “Validity and reliability of the research instrument; how to test the validation of a questionnaire/survey in a research”. In: *International Journal of Academic Research in Management* 5.3 (2016). Available on SSRN, posted 2018, pp. 28–36.
- [23] Clark Barrett et al. “CVC4”. In: *Computer aided verification* (2011), pp. 171–177. DOI: 10.1007/978-3-642-22110-1_14.
- [24] Armin Biere et al. *Handbook of Satisfiability*. Vol. 185. Frontiers in Artificial Intelligence and Applications. IOS Press, 2009. ISBN: 9781586039295.
- [25] Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. 3rd. Prentice Hall, 2010. ISBN: 9780136042594.

- [26] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. “Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)”. In: *Journal of the ACM* 53.6 (2006), pp. 937–977. DOI: 10.1145/1217856.1217859.
- [27] Cormac Flanagan et al. “Extended Static Checking for Java”. In: *Proceedings of the ACM SIGPLAN 2002 Conference on Programming language design and implementation*. ACM, 2002, pp. 234–245. DOI: 10.1145/512529.512558.
- [28] James C King. “Symbolic execution and program testing”. In: *Communications of the ACM* 19.7 (1976), pp. 385–394. DOI: 10.1145/360248.360252.
- [29] Cristian Cadar and Koushik Sen. “Symbolic execution for software testing: three decades later”. In: *Communications of the ACM* 56.2 (2013), pp. 82–90. DOI: 10.1145/2408776.2408795.
- [30] Armin Biere et al. “Symbolic model checking without BDDs”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 1999, pp. 193–207. DOI: 10.1007/3-540-49059-0_14.
- [31] Thomas H Cormen et al. *Introduction to algorithms*. 3rd. Standard reference for graph algorithms including topological sorting. MIT press, 2009. ISBN: 9780262033848.
- [32] Askalot Project. *QML Syntax Reference*. Accessed: 2025-11-30. 2025. URL: <https://docs.askalot.io/guide/qml-syntax/>.
- [33] Andrew W Appel and Jens Palsberg. *Modern Compiler Implementation in Java*. 2nd. Cambridge University Press, 2002. ISBN: 9780521820608.
- [34] Daniel Kroening and Michael Tautschnig. “CBMC–C bounded model checker”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. TACAS 2014. Springer, 2014, pp. 389–391. DOI: 10.1007/978-3-642-54862-8_26.
- [35] Alfred V Aho et al. *Compilers: principles, techniques, and tools*. 2nd. Addison Wesley, 2006. ISBN: 9780321486813.

Appendix A

Practical Examples of Complex Type Questionnaires

This appendix provides complete QML examples demonstrating the complex type concepts discussed in Section 3.7 and Section 4.5. These examples illustrate vector and matrix item types with their associated constraints in realistic survey scenarios.

A.1 Employee Satisfaction Survey

Consider a survey measuring employee satisfaction across multiple dimensions:

```
qmlVersion: "1.0"
questionnaire:
  title: "Employee Satisfaction Survey"
  codeInit: |
    # Track problematic areas
    problem_areas = []

  blocks:
    - id: satisfaction_assessment
      items:
        # Item I_1: Rate 5 aspects of job satisfaction
        # Type: Vector[5], Domain: Each element in [1,10]
        - id: job_aspects
          kind: QuestionGroup
          title: "Rate your satisfaction with each aspect"
          questions:
            - "Work-life balance"
            - "Compensation and benefits"
            - "Career development"
            - "Management support"
            - "Work environment"
          input:
            control: Slider
            min: 1
            max: 10
          codeBlock: |
            # Identify problem areas (rated below 5)
            aspect_names = ["Work-life", "Compensation", "Career", "Management", "
              Environment"]
            problem_areas = [aspect_names[i] for i in range(5)
              if job_aspects.outcome[i] < 5]
```

```
# Item I_2: Detailed questions if any aspect rated below 5
- id: followup_needed
  kind: Comment
  title: "Additional feedback needed"
  precondition:
    - predicate: any([job_aspects.outcome[j] < 5 for j in range(5)])
  codeBlock: |
    count = len(problem_areas)
    message = f"We identified {count} areas needing improvement:\n"
    for area in problem_areas:
        message += f"- {area}\n"
    print(message)

# Item I_3: Rank problematic areas by urgency
- id: problem_ranking
  kind: QuestionGroup
  title: "Rank these problematic areas by urgency"
  precondition:
    # Same condition as I_2
    - predicate: any([job_aspects.outcome[j] < 5 for j in range(5)])
  questions:
    - "First priority"
    - "Second priority"
    - "Third priority"
  input:
    control: Dropdown
    labels:
      1: "Work-life balance"
      2: "Compensation and benefits"
      3: "Career development"
      4: "Management support"
      5: "Work environment"
  postcondition:
    # Only allow ranking of actual problem areas
    - predicate: all([problem_ranking.outcome[i] in [j+1 for j in range(5)
        if job_aspects.outcome[j] < 5] for i in range(len(problem_ranking.outcome))
        if problem_ranking.outcome[i] > 0])
    hint: "Please only rank areas you rated below 5"

# Item I_4: Allocate 100 points for improvement priority
- id: improvement_allocation
  kind: QuestionGroup
  title: "Allocate 100 points across areas for improvement"
  precondition:
    - predicate: any([job_aspects.outcome[j] < 5 for j in range(5)])
  questions:
    - "Points for Work-life balance"
    - "Points for Compensation"
    - "Points for Career development"
    - "Points for Management"
    - "Points for Work environment"
  input:
    control: Editbox
    min: 0
    max: 100
  postcondition:
    - predicate: sum(improvement_allocation.outcome) == 100
    hint: "Points must sum to exactly 100"
    - predicate: all([improvement_allocation.outcome[j] >= 0 for j in range(5)])
    hint: "All values must be non-negative"
```

Code A.1: Multi-dimensional satisfaction survey

This example demonstrates:

- Vector items for multi-dimensional ratings using QuestionGroup
- Conditional logic based on vector components in preconditions
- Allocation constraint with sum requirement in postconditions
- Dynamic content generation using codeBlock

A.2 Product Comparison Matrix

A market research questionnaire comparing products:

```
qmlVersion: "1.0"
questionnaire:
  title: "Product Comparison Study"
  codeInit: |
    # Track best products per attribute
    best_per_attribute = []

  blocks:
    - id: comparison
      items:
        # Item I_1: Compare 4 products on 6 attributes
        # Type: Matrix[4][6], Domain: Each element in [0,100]
        - id: product_comparison
          kind: MatrixQuestion
          title: "Rate each product on these attributes (0-100)"
          rows:
            - "SmartPhone X"
            - "SmartPhone Y"
            - "SmartPhone Z"
            - "SmartPhone W"
          columns:
            - "Battery life"
            - "Screen quality"
            - "Camera"
            - "Performance"
            - "Build quality"
            - "Value for money"
          input:
            control: Editbox
            min: 0
            max: 100
          codeBlock: |
            # Calculate best product per attribute
            best_per_attribute = []
            for col in range(6):
              scores = [product_comparison.outcome[row][col] for row in range(4)]
              best_idx = scores.index(max(scores))
              best_per_attribute.append(best_idx)

        # Item I_2: Follow-up if any product scores perfect
        - id: perfect_product
          kind: Question
          title: "You rated a product perfectly! Tell us why."
          precondition:
            # Check if any product has 100 on all attributes
            - predicate: |
                any([product_comparison.outcome[j][k] == 100
                     for k in range(6)] for j in range(4))
          input:
            control: Radio
            labels:
              1: "It meets all my needs"
              2: "Best in its category"
              3: "Exceptional value"
              4: "Other reason"

        # Item I_3: Display best products per attribute
        - id: best_summary
          kind: Comment
          title: "Analysis complete"
          codeBlock: |
            # Create summary of best products
            products = ["SmartPhone X", "SmartPhone Y", "SmartPhone Z", "SmartPhone W"]
            attributes = ["Battery", "Screen", "Camera", "Performance", "Build", "Value"]
            summary = "Best products by attribute:\n"
            for i in range(6):
              best_prod = products[best_per_attribute[i]]
              score = product_comparison.outcome[best_per_attribute[i]][i]
              summary += f"- {attributes[i]}: {best_prod} (score: {score})\n"
            print(summary)
```

Code A.2: Product comparison matrix

This demonstrates:

- Matrix items for two-dimensional data using `MatrixQuestion`
- Complex preconditions involving matrix aggregations (checking for perfect scores)
- Derived calculations from matrix data using `codeBlock`
- Dynamic analysis and reporting of results

A.3 Market Research Survey with Complex Dependencies

This example demonstrates a questionnaire combining both vector and matrix types with inter-item dependencies:

```
qmlVersion: "1.0"
questionnaire:
  title: "Market Research Survey"
  codeInit: |
    # Initialize variables for tracking
    weakest_idx = -1

  blocks:
    - id: product_evaluation
      items:
        # Item I_1: Rate 4 products on 3 attributes
        # Type: Matrix[4][3] - 4 products x 3 attributes
        # Domain: Each element in [1,5]
        - id: product_ratings
          kind: MatrixQuestion
          title: "Rate each product on these attributes"
          rows:
            - "Product A"
            - "Product B"
            - "Product C"
            - "Product D"
          columns:
            - "Price"
            - "Quality"
            - "Service"
          input:
            control: Radio
            labels:
              1: "Poor"
              2: "Below Average"
              3: "Average"
              4: "Good"
              5: "Excellent"

        # Item I_2: Which product has best overall score?
        # Type: Scalar, Domain: [1,4] (product selection)
        - id: best_product
          kind: Question
          title: "Which product has the best overall score?"
          precondition:
            # At least one product must score 6+ total
            - predicate: |
                any([sum([product_ratings.outcome[i][j]
                        for j in range(3)]) >= 6
                    for i in range(4)])
          input:
            control: Radio
            labels:
              1: "Product A"
              2: "Product B"
              3: "Product C"
              4: "Product D"

        # Item I_3: Rank attributes by importance
        # Type: Vector[3] - permutation ranking
        - id: attribute_ranking
          kind: QuestionGroup
          title: "Rank these attributes by importance"
          precondition:
            # Only if chose Product A or B
            - predicate: best_product.outcome == 1 or best_product.outcome == 2
          questions:
            - "Price"
            - "Quality"
            - "Service"
          input:
            control: Dropdown
            labels:
              1: "Most important"
```

```

2: "Second most important"
3: "Least important"
postcondition:
# Ensure valid ranking (all different)
- predicate: len(set(attribute_ranking.outcome)) == 3
hint: "Each attribute must have a unique rank"

# Item I_4: Rate chosen product's weakest attribute
# Type: Scalar, Domain: [1,10]
- id: weakest_rating
kind: Question
title: "Rate improvement priority for the weakest attribute"
precondition:
# Find and check if weakest attribute exists
- predicate: |
min([product_ratings.outcome[best_product.outcome-1][j]
for j in range(3)]) < 4
codeBlock: |
# Identify and display the weakest attribute
prod_idx = best_product.outcome - 1
scores = [product_ratings.outcome[prod_idx][j] for j in range(3)]
weakest_idx = scores.index(min(scores))
attr_names = ["Price", "Quality", "Service"]
print(f"The weakest attribute is: {attr_names[weakest_idx]} (score: {
scores[weakest_idx]})")
input:
control: Slider
min: 1
max: 10

# Item I_5: Calculate weighted satisfaction scores
# Type: Vector[3] - weighted scores per attribute
- id: weighted_scores
kind: Comment
title: "Weighted satisfaction analysis completed"
codeBlock: |
# Calculate weighted scores based on importance ranking
prod_idx = best_product.outcome - 1
weighted = []
for j in range(3):
# Higher weight for more important attributes
weight = 4 - attribute_ranking.outcome[j]
score = product_ratings.outcome[prod_idx][j] * weight
weighted.append(score)
# Display results using built-in print function
attr_names = ["Price", "Quality", "Service"]
result = "Weighted scores:\n"
for j in range(3):
result += f"- {attr_names[j]}: {weighted[j]}\n"
print(result)

```

Code A.3: Questionnaire with complex type dependencies

This example demonstrates:

- Combining matrix ratings with scalar selection and vector ranking
- Preconditions that reference matrix elements using dynamic indices (e.g., `product_ratings.outcome[best_product.outcome-1][j]`)
- Postconditions enforcing permutation constraints on rankings
- Code blocks performing weighted calculations across complex types
- Multi-level dependencies: I_2 depends on I_1 , I_3 depends on I_2 , I_4 depends on both I_1 and I_2 , and I_5 depends on I_1 , I_2 , and I_3